MÓDULO 1. CRIPTOGRAFÍA (Unidad 3)

PREMÁSTER EN

ANÁLISIS DE MALWARE, REVERSING Y BUG BOUNTY



Campus Internacional CIBERSEGURIDAD

ENIIT INNOVA IT BUSINESS SCHOOL

UCAM Universidad Católica de Murcia

1. INTRODUCCIÓN 4
1.1. Criptosistemas de cifra de clave pública o asimétrica . . . 4
1.2. Funciones unidireccionales o de un solo sentido . . . 7
1.3. Protocolo de envoltura digital y criptosistema híbrido . . . 7
1.4. Principales servicios de seguridad de los criptosistemas de clave pública . . . 9
1.5. Inconvenientes de los sistemas de clave pública . . . 9
1.6. Principales algoritmos de clave pública . . . 10
1.7. Resumen . . . 11

2. Criptosistemas basados en el problema del Logaritmo Discreto 12
2.1. Fundamentos matemáticos . . . 13
2.1.1. Aritmética modular . . . 13
2.1.2. Grupos y Grupos finitos . . . 14
2.1.3. Grupos y subgrupos cíclicos . . . 15
2.1.4. Anillos . . . 18
2.1.5. Cuerpos finitos de Galois . . . 19
2.1.6. El problema del Logaritmo discreto . . . 20
2.2. Intercambio de claves de Diffie-Hellman . . . 22
2.2.1. Funcionamiento del Intercambio de Claves de Diffie-Hellman en $\mathbb{Z}_p^*$ . . . 23
2.2.2. Seguridad del Intercambio de claves Diffie-Hellman . . . 25
2.2.3. Ejemplo de uso del protocolo DHKE . . . 26
2.3. Criptosistema de ElGamal . . . 26
2.3.1. Fase de Generación de claves . . . 27
2.3.2. Fase de Cifrado . . . 27
2.3.3. Fase de Descifrado . . . 27
2.3.4. Seguridad . . . 28
2.4. Algoritmo de Firma digital (DSA) . . . 28
2.5. Resumen . . . 28

3. Criptosistema basado en la factorización de enteros: el RSA 29
3.1. Teoría de números . . . 29
3.1.1. Teoría de la divisibilidad y números primos . . . 29
3.1.2. Primalidad . . . 30
3.1.3. Algoritmo de Euclides . . . 31
3.1.4. Algoritmo Extendido de Euclides . . . 31
3.1.5. Función ϕ de Euler . . . 32
3.1.6. Teorema de Euler y Pequeño Teorema de Fermat . . . 33
3.2. Criptosistema RSA . . . 34
3.2.1. Generación de claves del RSA . . . 35
3.2.2. Cifrado de mensajes RSA . . . 35
3.2.3. Descifrado de mensajes RSA . . . 36
3.2.4. Generación de claves y descifrado con RSA-CRT . . . 37
3.2.5. Seguridad del RSA . . . 38
3.3. Resumen . . . 40

4. Criptografía con curvas elípticas 42
4.1. Algunos aspectos matemáticos 42
4.2. El problema del logaritmo discreto con curvas elípticas 46
4.3. Intercambio de clave de Diffie-Hellman con curvas elípticas (ECDH) 47
4.4. Criptosistemas de curvas elípticas: ECC 48
4.5. Resumen 48

Referencias 50

1. INTRODUCCIÓN

La protección de la información sensible contra accesos no autorizados o cambios fraudulentos, ha sido motivo de preocupación a través de los siglos. Las técnicas modernas de comunicación utilizan diversos dispositivos (ordenadores, tables, móviles) conectados a través de diversas redes, de modo que los datos que se transmiten a través de ellas son más vulnerables a estas amenazas. En los sistemas de cifrado simétricos estudiados en el tema anterior, para cifrar un mensaje es necesario que tenga lugar un intercambio de la clave privada entre quien envía el mensaje y quien lo recibe. Este intercambio de las claves privadas origina un problema insuperable: es muy difícil mantener el secreto. En este tema presentamos los elementos fundamentales de los sistemas de cifrado de clave pública, que resuelven este inconveniente de la criptografía simétrica. En primer lugar se analizan las debilidades de los sistemas de cifrado simétrico, algunos conceptos básicos necesarios para introducir la criptografía de clave pública y las principales funciones de seguridad de estos criptosistemas. A continuación se tratan brevemente los inconvenientes que estos presentan y los principales algoritmos que emplean los criptosistemas de clave pública más utilizados en la actualidad. Para profundizar más en este tema y obtener referencias adicionales podremos consultar la bibliografía referenciada en este documento.

La criptografía de clave secreta ha sido utilizada por al menos 4000 años a diferencia de la criptografía de clave pública que es bastante nueva y se introdujo en 1976 por Whitfield Diffie, Martin Hellman y Ralph Merkle. Revisemos las propiedades básicas de la criptografía simétrica: en primer lugar se utiliza la misma clave secreta para codificar y decodificar; en segundo lugar los algoritmos de cifrado y descifrado son muy similares. Los algoritmos simétricos modernos tales como AES son muy seguros y rápidos. Sin embargo, los criptosistemas convencionales que utilizan estos algoritmos han presentado dos desventajas esenciales en los sistemas de comunicación modernos:

- El problema de la autenticación. En un sistema de comunicación controlado por computadoras se requiere el equivalente electrónico de una firma. Los criptosistemas simétricos no poseen esta característica de forma natural, de modo que cuando existe un conflicto entre el receptor y el emisor de un mensaje, resulta imposible determinar quién tiene la razón. Por ejemplo, en aplicaciones de comercio electrónico es importante probar que el emisor realmente envió un determinado mensaje, por ejemplo, una orden en línea para la compra de un televisor. Si el emisor cambia de parecer, podrá afirmar que el vendedor ha generado falsamente la orden de compra electrónica. Evitar esto es importante, y se denomina principio de no-repudio, que puede lograrse con la criptografía asimétrica, como se verá más adelante en el tema Firmas Digitales.

- El problema de la gestión y distribución de claves: Un sistema de comunicación con n usuarios que utilizan un sistema criptográfico convencional para comunicarse entre ellos implica la necesidad de $\binom{n}{2}$ claves y la misma cantidad de canales seguros. Cada vez que un usuario quiera cambiar sus claves, o un nuevo usuario quiera participar en el sistema, tendrán que generarse y distribuirse, $n-1$ nuevas claves y canales seguros respectivamente.

1.1. CRIPTOSISTEMAS DE CIFRA DE CLAVE PÚBLICA O ASIMÉTRICA

En 1976 Whitfield Diffie y Martin Hellman de la Universidad de Stanford, propusieron un método radicalmente nuevo para la distribución de claves, conocido como *Intercambio de claves Diffie-Hellman* en su artículo "New Directions in Cryptography" [1]. En este artículo los autores presentan

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

las nociones básicas de la Criptografia en clave pública, y algunos problemas que ésta resuelve. Se menciona también el trabajo de Ralph Merkle, de la Universidad de California en Berkeley, quien había llegado a la misma idea de clave pública que ellos.

De este modo, Diffie, Hellman y Merkle [2], hicieron una propuesta revolucionaria basada en la siguiente idea: No es necesario que la clave utilizada por la persona que cifra el mensaje sea secreta. La parte crucial es que el receptor sólo pueda descifrarlo usando una clave secreta. Para ello utilizaron un par de claves diferentes: una pública conocida por todos y otra privada en poder de una única persona, con la particularidad de que es computacionalmente difícil deducir la clave privada a partir de la clave pública. Cualquier persona con la clave pública puede cifrar un mensaje pero no descifrarlo. Sólo la persona con la clave privada correspondiente puede descifrar el mensaje.

Matemáticamente, el proceso se basa en las *funciones trampa unidireccionales*. El cifrado es la dirección fácil. Cualquiera puede cifrar un mensaje. El descifrado es la dirección difícil. El secreto, o trampilla, es la clave privada. Con ese secreto, el descifrado es tan fácil como el cifrado. Así es como Alice puede enviar un mensaje a Bob usando la criptografía de clave pública:

1. Alice y Bob acuerdan el uso de un criptosistema de clave pública.
2. Bob envía a Alice su clave pública.
3. Alice cifra su mensaje usando la clave pública de Bob y se lo envía.
4. Bob descifra el mensaje de Alice usando su clave privada.

Observe cómo la criptografía de clave pública resuelve el problema de la gestión de claves de los criptosistemas simétricos. Antes, Alice y Bob tenían que ponerse de acuerdo sobre la clave en secreto. Alice podía elegir una clave al azar, pero tendría que enviarla a Bob; podría entregársela a él en algún momento de antemano, lo que requiere previsión; o podría enviarla usando correo seguro, lo que consume tiempo. La criptografía de clave pública lo hace fácil. Sin arreglos previos, Alice puede enviar un mensaje seguro a Bob. Eve una intrusa presente en todo el intercambio, tiene la clave pública de Bob y un mensaje cifrado con esa clave, pero no puede recuperar la clave privada de Bob ni el mensaje.

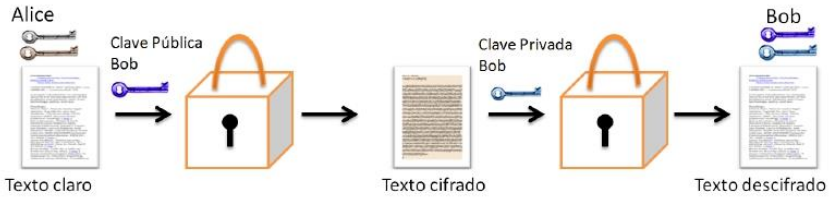


Figura 1: Esquema genérico de un cifrado asimétrico.

La Fig. 1 se muestra el esquema genérico de un criptosistema de clave pública. Una definición general de los sistemas de clave pública es la siguiente [3]:

> Dado $\mathcal{M}$ el conjunto de todos los posibles mensajes, $\mathcal{C}$ el de todos los mensajes cifrados y $\mathcal{K}$ el conjunto de todas las claves posibles. Un *criptosistema de clave pública* es un par de familias de funciones, $(\mathcal{C}, \mathcal{D})$, una de cifrado, $\mathcal{C}$, y otra de descifrado, $\mathcal{D}$, con claves de cifrado $e \in \mathcal{K}$, y

descifrado $d \in \mathcal{K}$, de modo que para cada $M \in \mathcal{M}$ se cumple que:

$$\mathcal{D}_d(\mathcal{C}_e(M)) = \mathcal{D}_d(C) = M,$$
$$\mathcal{D}_e(\mathcal{C}_d(M)) = \mathcal{D}_e(C') = M.$$

Además, debe cumplirse que conocida la función de cifrado y la clave pública, $\mathcal{C}_e$ y conocido un texto cifrado aleatorio $C \in \mathcal{C}$, sea computacionalmente imposible hallar un mensaje $M \in \mathcal{M}$ tal que: $\mathcal{C}_e(M) = C$.

En la práctica, los usuarios que desean comunicarse se ponen de acuerdo y eligen un criptosistema de clave pública determinado. Cada usuario tiene su propia clave pública y su clave privada; las claves públicas se publican en una base de datos para tal efecto. El hecho de tener disponible una clave pública garantiza que las personas involucradas en la comunicación omitan el intercambio de claves a través de canales inseguros.

- Alice obtiene la clave pública de Bob de la base de datos.
- Alice cifra su mensaje usando la clave pública de Bob y se lo envía a este.
- Bob descifra el mensaje de Alice usando su clave privada.

En el primer protocolo, Bob tuvo que enviarle a Alice su clave pública antes de que pudiera enviarle un mensaje. El segundo protocolo se parece más al correo tradicional. Bob no se involucra en el protocolo hasta que quiera leer su mensaje. El cifrado de clave asimétrica utiliza un par de claves relacionadas matemáticamente, en el que una de ellas, la clave privada, descifra el mensaje cifrado que se ha realizado con la clave pública correspondiente. Algunos (pero no todos) de estos algoritmos poseen la propiedad adicional de que una de las claves del par no se puede deducir de la otra por ningún método conocido que no sea por prueba exhaustiva. Mientras la clave privada permanezca en secreto, la clave pública puede ser conocida públicamente durante mucho tiempo sin comprometer la seguridad, haciendo que sea seguro reutilizar el mismo par de claves de forma indefinida.

En el cifrado asimétrico las operaciones de cifrado y descifrado se realizan utilizando diferentes claves –una clave pública, conocida por todos y una clave privada–. También se conoce como cifrado de clave pública.

Las parejas de claves tienen funciones diversas y muy importantes, entre las que destacan:

- Cifrar la información.
- Asegurar la integridad de los datos transmitidos.
- Garantizar la autenticidad del emisor.

La efectividad de los algoritmos asimétricos depende de una clase de problemas matemáticos conocidos como *funciones de un solo sentido o unidireccionales*, que requieren relativamente poca potencia de cálculo en una dirección, en cambio se necesita muchos recursos para calcular la inversa.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

1.2. FUNCIONES UNIDIRECCIONALES O DE UN SOLO SENTIDO

Una función $f(\cdot)$ es una función de un solo sentido si:

- $y = f(x)$ es computacionalmente fácil, y
- $x = f^{-1}(y)$ es computacionalmente inviable.

En términos matemáticos, una función es fácil de calcular si se puede evaluar en tiempo polinomial, es decir, si su tiempo de ejecución es una expresión polinomial. Para ser empleado en esquemas criptográficos prácticos, el cálculo de la función $y = f(x)$ deberá ser lo suficientemente rápido con vistas a que que no produzca tiempos de ejecución inaceptablemente lentos en una aplicación. El cálculo de la función inversa $x = f^{-1}(y)$ deberá ser complicada computacionalmente, de modo que no sea factible evaluarla en un período de tiempo razonable, por ejemplo, 10.000 años, cuando se utiliza el mejor algoritmo conocido.

Un ejemplo clásico de función unidireccional es la multiplicación de números primos grandes. Es bastante rápido multiplicar dos primos grandes, pero muy difícil factorizar el producto de dos primos grandes. Debido a las propiedades matemáticas de las funciones unidireccionales, la mayor parte de las claves posibles tienen poca calidad para su uso criptográfico; solo una pequeña parte de las claves posibles de una cierta longitud son candidatas ideales, y por tanto los algoritmos asimétricos requieren claves muy largas para alcanzar el mismo nivel de seguridad proporcionado por las claves simétricas, relativamente más cortas.

Las exigencias de generar el par de claves y realizar el cifrado/descifrado hacen que los algoritmos asimétricos sean costosos computacionalmente. Los algoritmos simétricos pueden usar como clave una secuencia pseudoaleatoria criptográficamente segura que se puede generar rápidamente, siendo ideales para generar una clave de sesión desechable para uso a corto plazo. Por consiguiente, es una práctica común utilizar un sistema criptográfico asimétrico, cuyas claves son suficientemente largas para ser seguras, para intercambiar una clave simétrica desechable mucho más corta (pero igualmente segura). El algoritmo asimétrico –lento– envía de forma segura una clave simétrica de sesión, y entonces el algoritmo simétrico –más rápido–, toma el control para el resto del mensaje. A continuación presentamos un protocolo basado en ambos sistemas.

1.3. PROTOCOLO DE ENVOLTURA DIGITAL Y CRIPTOSISTEMA HÍBRIDO

El *protocolo de envoltura digital* permite que los usuarios Alice, (A), y Bob, (B), intercambien mensajes secretos utilizando un criptosistema de clave secreta, $(\mathcal{E}, \mathcal{E}^{-1})$, y otro de clave pública, $(\mathcal{C}, \mathcal{D})$, acordado con anterioridad [3].

Protocolo 1.1 Cifrado con envoltura digital

1. B elige aleatoriamente una clave k para el criptosistema de clave secreta acordado.
2. B cifra el mensaje, M con la clave k mediante el criptosistema de clave secreta, $\mathcal{E}_k(M) = C$.
3. B cifra la clave secreta k, mediante el criptosistema de clave pública elegido, usando la clave pública de A, e: $\mathcal{C}_e(k) = K$.
4. B envía a A el el texto cifrado C y la clave secreta cifrada K.

Observación 1.1 *Cada vez que utilicemos la palabra* ***aleatorio*** *en el texto, nos referimos a que el número fue elegido con la ayuda de un generador de números aleatorios (o pseudoaleatorios) criptográficamente seguro, es decir, un algoritmo que genera una secuencia de números o bits impredecible, y que tiene todas las propiedades estadísticas de una secuencia verdaderamente aleatoria.*

Para cifrar un mensaje Bob ejecuta el Protocolo 1.1 y lo envia a Alice. Para descifrar el mensaje Alice utiliza el Protocolo (1.2), mostrado a continuación.

Protocolo 1.2 Descifrado con envoltura digital

1. A obtiene la clave secreta, k, descifrando K usando su clave privada, d, correspondiente al criptosistema de clave pública empleado: $\mathcal{D}_d(K) = \mathcal{D}_d(\mathcal{C}_e(k)) = k$.
2. A descifra el mensaje mediante el criptosistema de clave secreta usando la clave secreta descifrada: $\mathcal{E}_k^{-1}(C) = \mathcal{E}_k^{-1}(\mathcal{E}_k(M)) = M$.

(Criptosistema híbrido) es un criptosistema de clave pública que se emplea para enviar una *clave de sesión* o *clave efímera* cifrada, que luego será utilizada en un criptosistema de clave secreta. Estos criptosistemas aprovechan las ventajas que ofrecen los criptosistemas de clave simétrica como los de clave asimétrica, usando lo mejor de cada uno de ellos.

Consta de dos etapas: a) encapsulación de claves (KEM, *Key Encapsulation Mechanism*); b) encapsulación de datos (DEM, *Data Encapsulation Mechanism*) [4].

ENCAPSULACIÓN DE CLAVE (KEM)

Esta etapa consta de los elementos siguientes:

1. `KEM.KeyGen`, algoritmo de generación de claves pública y privada (PK, SK).
2. `KEM.Encrypt`, algoritmo de cifrado cuya entrada es una clave pública PK y un conjunto de opciones, opt, la salida es el par formado por una clave K y su cifrado correspondiente, C_0, usando la clave pública PK.
3. `KEM.Decrypt`, algoritmo de descifrado cuya entrada es una clave privada, SK, y la clave cifrada C_0. La salida es la clave K.
4. `KEM.OutputKeyLen`, es un entero positivo, que da cuenta de la longitud de la clave K.

ENCAPSULACIÓN DE DATOS (DEM)

1. `DEM.Encrypt`, algoritmo de cifrado cuya entrada es la clave K, una etiqueta L y un mensaje M. La salida es el texto cifrado C_1.
2. `DEM.Decrypt`, algoritmo de descifrado cuya entrada está formada por K, L y C_1. La salida es el mensaje M.
3. K, L, M y C_1 son cadenas de bytes, tales que L y M pueden ser de longitudes variables y K tiene longitud `DEM.KeyLen`.

1.4. PRINCIPALES SERVICIOS DE SEGURIDAD DE LOS CRIPTOSISTEMAS DE CLAVE PÚBLICA

Los criptosistemas de clave pública pueden utilizarse para realizar otras tareas de seguridad además de cifrar datos. Los principales servicios de seguridad de los algoritmos de clave pública se enumeran a continuación:

- Establecimiento de claves. Existen protocolos para establecer claves secretas sobre un canal inseguro. Ejemplos de tales protocolos incluyen el intercambio de claves Diffie-Hellman (DHKE) o protocolos de transporte de clave RSA.
- No-repudio. Se puede lograr el no-repudio y la integridad del mensaje con los algoritmos de firma digital, por ejemplo, RSA, DSA o ECDSA.
- Identificación. Podemos identificar entidades que utilizan protocolos de desafío y respuesta a través de sus firmas digitales, por ejemplo, en aplicaciones como tarjetas bancarias inteligentes o teléfonos móviles.
- Cifrado. Podemos cifrar mensajes usando algoritmos como RSA o ElGamal.

Las funciones de identificación y cifrado también pueden lograrse con sistemas simétricos, pero normalmente requieren mucho más esfuerzo con la gestión de claves.

1.5. INCONVENIENTES DE LOS SISTEMAS DE CLAVE PÚBLICA

El problema de la lentitud: En la práctica el principal inconveniente de los algoritmos de clave pública es que el cifrado de datos es computacionalmente intenso, lo que consume mucho tiempo y los hace muy lentos. Muchos cifrados de bloque y de flujo pueden cifrar alrededor de cien a mil veces más rápido que los algoritmos de clave pública. Por lo tanto, irónicamente la criptografía de clave pública rara vez se utiliza para el cifrado real de los datos. Por otro lado, los algoritmos simétricos no proveen las funcionalidades de no-rechazo y el intercambio o establecimiento de claves es muy costoso.

Es por eso que la mayoría de los protocolos prácticos son híbridos, que utilizan una combinación de los algoritmos de clave privada y de clave pública. Por ejemplo, SSL/TLS (*Secure Sockets Layer/Transport Layer Security*) son protocolos que hacen uso de certificados digitales para establecer comunicaciones seguras a través de Internet, o IPsec (*Internet Protocol security*) que es un conjunto de protocolos cuya función es asegurar las comunicaciones sobre el Protocolo de Internet (IP) autenticando y/o cifrando los datos enviados en la red.

El problema de la autenticidad de las claves públicas: Una ventaja importante de la criptografía asimétrica es que podemos distribuir libremente las claves públicas. Sin embargo, en la práctica, las cosas son un poco más difíciles porque todavía tenemos que asegurar su autenticidad. ¿Cómo sabemos realmente que una cierta clave pública pertenece a una determinada persona? Este problema a menudo se resuelve usando certificados digitales. A grandes rasgos, los certificados digitales vinculan una clave pública a una cierta identidad. Este es un problema importante en muchas aplicaciones de seguridad, por ejemplo, al realizar transacciones de comercio electrónico en Internet. Este tema será tratado con mayor detalle en la clase sobre Certificados digitales.

El problema de la longitud de las claves: Los algoritmos de clave pública requieren claves muy largas, lo que contribuye a que los tiempos de ejecución sean lentos. Los algoritmos de clave pública

Cuadro 1.1: Longitudes de clave y niveles de seguridad de los algoritmos.

Criptosistemas	Niveles de Seguridad			
Clave secreta	80	128	192	256
RSA: Factorización de enteros	1024	3072	7680	15360
DH, DSA, ElGamal: Logaritmo discreto	1024	3072	7680	15360
ECDH, ECDSA: Curvas elípticas	160	256	384	512

se basan en funciones de la teoría de números. Una característica distintiva de ellos es que requieren cálculos cuyos operandos son muy largos. Mientras más grandes sean los operandos y las claves, más seguros se vuelven los algoritmos. Con el fin de comparar diferentes algoritmos, a menudo se considera el nivel de seguridad.

Se dice que un algoritmo tiene un "nivel de seguridad de n bits" si el ataque más conocido requiere 2^n pasos. Los algoritmos simétricos con un nivel de seguridad n tienen una clave de longitud n bits. Sin embargo, la relación entre la fortaleza criptográfica y la seguridad no es tan directa en el caso asimétrico. El cuadro 1.1 muestra las longitudes de bits recomendadas para los algoritmos de clave pública para cuatro niveles de seguridad 80, 128, 192 y 256 bits. Puede verse que los esquemas similares a RSA y los esquemas de logaritmos discretos requieren operandos y claves muy largos. La longitud de la clave de los esquemas de curva elíptica es significativamente menor, pero al mismo tiempo es dos veces más larga que la obtenida de los cifrados simétricos con la misma fuerza criptográfica.

1.6. PRINCIPALES ALGORITMOS DE CLAVE PÚBLICA

Los criptosistemas de clave pública más empleados en la actualidad son:

1. El *criptosistema RSA* [5].
2. El *criptosistema de ElGamal* [6].
3. El *criptosistema con curvas elípticas* [7, 8].

Los algoritmos de clave pública pueden clasificarse en función del problema computacional que abordan en tres grupos principales (Figura 2):

- Esquema basado en la Factorización de Enteros: Estos algoritmos se basan en el hecho de la complejidad en la factorización de enteros grandes. El representante más importante de esta familia de algoritmos es el RSA.
- Esquema basado en el Logaritmo Discreto: Se basan en el problema del logaritmo discreto en cuerpos finitos. Los ejemplos más destacados incluyen el protocolo de Intercambio de Claves Diffie-Hellman, el criptosistema ElGamal o el Algoritmo de Firma Digital (DSA).
- Esquema basado en Curvas Elípticas (EC): Es la realización de los sistemas basados en el problema del logaritmo discreto con curvas elípticas. Los ejemplos más significativos son el protocolo de Intercambio de Claves con Curva Elíptica Diffie-Hellman (ECDH) y el Algoritmo de Firma Digital de Curva Elíptica (ECDSA).

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

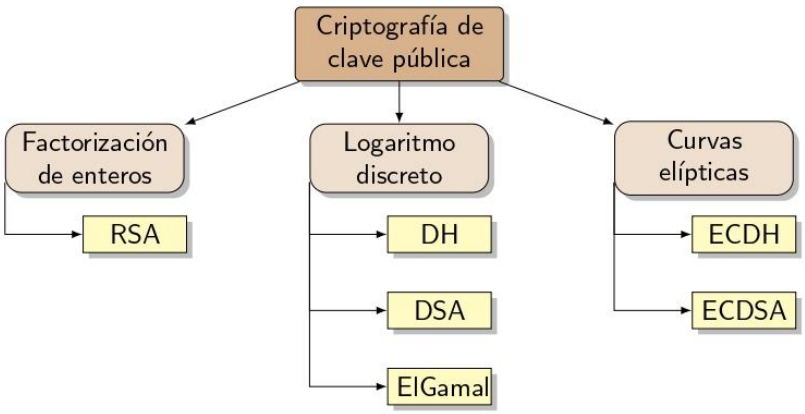


Figura 2: Clasificación de los criptosistemas asimétricos en función del problema computacional que abordan.

Es importante señalar que cada una estas tres familias de algoritmos puede utilizarse para proporcionar los mecanismos de establecimiento de claves, el no-rechazo a través de firmas digitales y el cifrado de datos. Para poder comprender el funcionamiento de estos algoritmos y su utilización de manera segura, es necesario dominar las herramientas matemáticas que los sostienen, es por esta razón que estudiaremos de manera práctica, los principales conceptos y herramientas matemáticas involucradas, desde el punto de vista ingenieril.

1.7. RESUMEN

La criptografía de clave pública se caracteriza por utilizar un par de claves para garantizar la autenticidad, confidencialidad y seguridad de las comunicaciones. Las dos claves pertenecen a la misma persona o entidad que utilizará el canal de comunicación inseguro. Una clave es pública y se puede entregar a cualquier persona, la otra clave es privada y el propietario debe guardarla de modo que nadie tenga acceso a ella. Además, los métodos criptográficos garantizan que esa pareja de claves sólo se pueda generar una vez, de modo que se puede asumir que no es posible que dos personas hayan obtenido casualmente la misma pareja de claves.

Si el remitente usa la clave pública del destinatario para cifrar un mensaje, una vez cifrado, sólo la clave privada del destinatario podrá descifrar este mensaje, ya que es el único que la conoce. Por tanto se logra la *confidencialidad* del envío del mensaje, nadie salvo el destinatario puede descifrarlo.

Si el propietario del par de claves usa su clave privada para cifrar el mensaje, cualquiera puede descifrarlo utilizando su clave pública. En este caso se consigue por tanto la *identificación y autenticación* del remitente, ya que se sabe que sólo pudo haber sido él quien empleó su clave privada (salvo que alguien se la hubiese robado). Esta idea es el fundamento de la firma electrónica.

Los sistemas de cifrado de clave pública solucionan en gran medida el problema del intercambio de claves de los sistemas de cifrado simétricos. Con las claves públicas no es necesario que el

remitente y el destinatario se pongan de acuerdo en la clave a emplear. Todo lo que se requiere es que, antes de iniciar la comunicación secreta, el remitente consiga una copia de la clave pública del destinatario. La clave pública puede ser usada por cualquiera que desee comunicarse con su propietario. Por tanto, se necesitarán sólo n pares de claves por cada n personas que deseen comunicarse entre sí.

La efectividad de los algoritmos asimétricos depende de problemas matemáticos conocidos como funciones unidireccionales, que requieren relativamente poca potencia de cálculo para ejecutarse en una dirección, pero muchísima potencia para calcular la inversa. Los algoritmos de clave pública son computacionalmente costosos (lentos), y por lo tanto no recomendables para el cifrado de grandes cantidades de datos, como por ejemplo, el cifrado de imágenes. Los sistemas de clave pública combinados con los sistemas de clave secreta pueden proporcionar todas las funcionalidades requeridas por los protocolos de seguridad modernos.

2. CRIPTOSISTEMAS BASADOS EN EL PROBLEMA DEL LOGARITMO DISCRETO

Los sistemas de cifrado actuales utilizan algoritmos basados en problemas matemáticos intratables o difíciles de resolver con la capacidad de cómputo actual cuando se trabaja con valores por encima de miles de bits.

La seguridad de muchas técnicas criptográficas de clave pública depende de lo intratable que resulta el problema del logaritmo discreto, por ejemplo el intercambio de claves de Diffie-Hellman y sus variantes, el sistema criptográfico ElGamal, así como el esquema de su firma digital y sus variantes.

Los grupos de mayor interés en criptografía son los grupos multiplicativos GF_p^* del cuerpo finito GF_p incluyendo los casos particulares del grupo multiplicativo $\mathbb{Z}_p^*$ de los enteros módulo un número primo p y el grupo multiplicativo $GF_{2^m}^*$ del cuerpo finito GF_{2^m} de característica 2. También es de sumo interés el grupo unidad $\mathbb{Z}_n^*$, donde n es un entero compuesto, el grupo de puntos en una curva elíptica sobre un cuerpo finito y el jacobiano de una curva hiperelíptica definida sobre un cuerpo finito.

En esta sección se presentan el algoritmo de cifrado asimétrico ElGamal y el protocolo de intercambio de claves de Diffie y Hellman, cuya seguridad se basa en *el problema del logaritmo discreto*, por esta razón es necesario revisar algunas definiciones, teoremas y proposiciones pertenecientes al campo del álgebra abstracta que nos permitan entender en qué consisten estos algoritmos.

Para profundizar más en este tema recomendamos consultar la bibliografía citada en este documento. Específicamente para los temas relacionados con el álgebra y la teoría de números recomendamos el libro "*A Computational Introduction to Number Theory and Algebra*" [9] que se encuentra disponible dentro de los materiales del curso.

El problema de logaritmo discreto puede definirse de la manera siguiente:

Definición 2.1 ***El Problema del Logaritmo Discreto en*** Z_p^**: Dados el grupo cíclico finito* Z_p^* *de orden* $p-1$*, un elemento primitivo* $\alpha \in Z_p^*$ *y otro elemento* $\beta \in Z_p^*$*. El problema del logaritmo discreto consiste en determinar el entero* $1 \leq x \leq p-1$ *, tal que:*

$$\alpha^x \equiv \beta \bmod p$$

2.1. FUNDAMENTOS MATEMÁTICOS

De la definición del problema del logaritmo discreto se derivan una serie de conceptos que pertenecen al álgebra abstracta: parte de las matemáticas que estudia las estructuras algebraicas como las de grupo, subgrupos, grupos finitos, grupos cíclicos y cuerpos que son esenciales para la comprensión de los algoritmos de clave pública basados en el logaritmo discreto. Además, en criptografía es habitual el uso de la aritmética modular debido a que el cálculo de logaritmos discretos y raíces cuadradas modulo p pueden ser problemas computacionalmente costosos; la aritmética modular se puede realizar cómodamente en ordenadores, ya que restringe tanto el rango de valores intermedios calculados como el resultado final: no se tienen que utilizar grandes números (y por tanto, grandes reservas de memoria) para almacenar los resultados, ya que su tamaño siempre estará limitado. Por esta razón revisamos muy brevemente la aritmética modular. Si desea profundizar en estos temas recomendamos revisar las referencias bibliográficas.

2.1.1. ARITMÉTICA MODULAR

La aritmética modular se conoce también como aritmética del reloj, debido a su parecido con la forma que tenemos de contar el tiempo. Por ejemplo, si son las 20:11:59 y pasa un segundo, decimos que son las 20:12:00, y no las 20:11:60. Como vemos, los segundos se expresan empleando sesenta valores cíclicamente, deforma que después del 59 le sigue el 0. Matemáticamente diríamos que los segundos se expresan módulo 60 (lo mismo vale para los minutos). Más formalmente:

Definición 2.2 *Definición de congruencia. Dado $p \in \mathbb{Z}$, $p > 1$, se dice que $a, b \in \mathbb{Z}$ son congruentes módulo p si y solo si $p|(a-b)$. Se utiliza la notación, $a \equiv b \bmod p$; donde p es el módulo de la congruencia.*

En esta relación, b es el residuo (o resto) de $a \bmod p$. Al conjunto de enteros desde 0 hasta $p-1$ se le denomina conjunto completo de residuos módulo p. Esto significa que para cada entero a, su residuo módulo p es un número entero comprendido entre 0 y $p-1$. La operación $a \bmod p$ se denomina reducción modular.

La relación de congruencia módulo p es una relación de equivalencia para todo $p \in \mathbb{Z}$ lo que quiere decir que verifican las propiedades reflexiva, simétrica y transitiva. Para abundar visita: Relación de equivalencia.

Es importante darse cuenta de que si p divide a $a-b$, esto supone que ambos a y b tienen el mismo resto al ser divididos por el módulo p.

Ejemplo 2.1 *Congruencias:* $-6 \equiv 1 \bmod 7$ *(porque* $7|-7$*) y* $41 \equiv 6 \bmod 7$ *(porque* $7|35$*).* ■

Ejemplo 2.2 *Encontrar dos números enteros distintos, cada uno los cuales sea congruente con 13 módulo 11. Sabemos que a cumple que:*

$$a \equiv 13 \bmod 11 \Longleftrightarrow a = 13 + 11q,$$

con $q \in \mathbb{Z}$. Si seleccionamos $q = -2$ y $q = 2$ tendremos que,

$$\begin{aligned} a &= 13 + 11(-2) = -9 \Rightarrow -9 \equiv 13 \bmod 11 \\ a &= 13 + 11(2) = 35 \quad \Rightarrow \ 35 \equiv 13 \bmod 11 \end{aligned}$$ ■

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

Teorema 2.1 *Sea p cualquier numero entero positivo. Entonces,*

a) Cualquier número entero es congruente modulo p exactamente con uno de los enteros $0, 1, \ldots, p-1$.

b) Dos números enteros son congruentes entre sí modulo p sí, y solo si ambos dan el mismo resto al dividirlos por p.

Definición 2.3 *Clases de Equivalencia. Dado un numero entero cualquiera a, su clase de equivalencia es el conjunto formado por todos los enteros que dan el mismo resto que a al dividirlos entre p.*

En general cualquier entero pertenecerá a alguna clase de equivalencia por lo que tendremos p clases de equivalencia módulo p, es decir, tendremos números congruentes con 0, números congruentes con 1, hasta números congruentes con $p-1$. Para abundar ver Sec. (1.6) y (1.7) en: EMA

2.1.2. GRUPOS Y GRUPOS FINITOS

Definición 2.4 ***Grupo**: Un conjunto $(\mathbb{G})$ con una operación $\circ$ definida sobre él, se dice que es:*

1. *Un **Grupo**, si $(\mathbb{G}, \circ)$ cumple la propiedad asociativa, existe un elemento neutro y cada elemento de $\mathbb{G}$ tiene un opuesto o inverso.*
2. *Un **Grupo Conmutativo o Abeliano**[a] si $(\mathbb{G}, \circ)$ es un grupo y además la operación $\circ$ es conmutativa.*

[a]El adjetivo Abeliano se utiliza para honrar a N.H. Abel (1802-1829), un excepcional matemático noruego, que utilizó Grupos para demostrar que generalmente, la ecuación polinómica de grado ≥ 5 no se puede resolver por radicales. El premio de matemáticas de mayor cuantía económica en el mundo, lo concede el gobierno noruego y se llama Abel.

Observación 2.1 *Muchos de los criptosistemas de clave pública utilizan el grupo abeliano, por ejemplo en la operación de suma de los criptosistemas curvas elípticas.*

Veamos ahora el grupo $\mathbb{Z}_n^*$ que es muy importante para muchas primitivas criptográficas como el intercambio de claves de Diffie-Hellman, los algoritmos de cifrado y de firma digital de ElGamal, y muchos otros.

Teorema 2.2 ***El conjunto** $\mathbb{Z}_p^*$ El conjunto $\mathbb{Z}_p^*$ está compuesto por todos los enteros, $i = 0, 1, ..., p-1$, para los que, $\mathrm{mcd}(i, p) = 1$ forma un* grupo abeliano *bajo la multiplicación modulo p. El elemento identidad es, $e = 1$.*

GRUPOS FINITOS

Las estructuras matemáticas que se emplean en criptografía son casi siempre finitas. Veamos el concepto de grupo finito.

Definición 2.5 ***Grupo finito****: Un grupo $(\mathbb{G}, \circ)$ es finito si tiene un número finito de elementos. El orden de un grupo finito $\mathbb{G}$ denotado por $|\mathbb{G}|$ es su cardinalidad, es decir el número de elementos que lo forman: $|\mathbb{G}| = Ord\mathbb{G} = Card\mathbb{G}$.*

Definición 2.6 ***Orden de un elemento****: El orden de un elemento $Ord(a)$ de un grupo cíclico $(\mathbb{G}, \circ)$ es el menor entero positivo k tal que $a^k = a \circ a \circ \ldots \circ a = 1$, donde 1 es el elemento de identidad de $\mathbb{G}$.*

Ejemplo 2.3 *Determinar el orden de $a = 3$ en el grupo $\mathbb{Z}_{11}^*$: Para ello calculamos las potencias de a hasta obtener el elemento de identidad 1.*

$$\begin{aligned} a^1 &= 3 \\ a^2 &= a \cdot a = 3 \cdot 3 = 9 \\ a^3 &= a^2 \cdot a = 9 \cdot 3 = 27 \equiv 5 \bmod 11 \\ a^4 &= a^3 \cdot a = 5 \cdot 3 = 15 \equiv 4 \bmod 11 \\ a^5 &= a^4 \cdot a = 4 \cdot 3 = 12 \equiv 1 \bmod 11 \end{aligned}$$

Por tanto, $Ord(3) = 5$ ■

Veamos qué sucede si continuamos multiplicando el resultado por a:

$$\begin{aligned} a^6 &= a^5 \cdot a = 1 \cdot a \equiv 3 \bmod 11 \\ a^7 &= a^5 \cdot a^2 = 1 \cdot a^2 \equiv 9 \bmod 11 \\ a^8 &= a^5 \cdot a^3 = 1 \cdot a^3 \equiv 5 \bmod 11 \\ a^9 &= a^5 \cdot a^4 = 1 \cdot a^4 \equiv 4 \bmod 11 \end{aligned} \qquad \begin{aligned} a^{10} &= a^5 \cdot a^5 = 1 \cdot 1 \equiv 1 \bmod 11 \\ a^{11} &= a^{10} \cdot a = 1 \cdot a \equiv 3 \bmod 11 \\ &\vdots \end{aligned}$$

Comprobamos que a partir de este punto, las potencias de a mantienen la secuencia $3, 9, 5, 4, 1, \ldots$ indefinidamente, siguiendo un comportamiento cíclico.

2.1.3. GRUPOS Y SUBGRUPOS CÍCLICOS

Los grupos que pueden ser generados por un único elemento se llaman Grupos cíclicos; es decir, hay un elemento α del grupo $\mathbb{G}$ (llamado "generador" de $\mathbb{G}$), tal que todo elemento de $\mathbb{G}$ puede ser expresado como una potencia de α. El hecho de tener un único elemento como generador del grupo facilita el trabajo con los grupos. Como veremos más adelante los grupos multiplicativos de los cuerpos finitos son todos cíclicos, lo que tiene gran importancia en criptografía.

Definición 2.7 ***Grupos cíclicos****: Un grupo $\mathbb{G}$ es cíclico si existe un elemento $\alpha \in \mathbb{G}$ tal que para cada $b \in \mathbb{G}$ existe un entero i siendo $b = \alpha^i$. Al elemento α se le denomina generador o elemento primitivo de G.*

De esta definición se deduce que un grupo formado por todas las potencias de uno de sus elementos es un grupo cíclico. Es decir, si un elemento b de un grupo $\mathbb{G}$ se puede escribir como una potencia $\alpha^i = b$ de dicho elemento para i, entonces α genera al grupo entero.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

Proposición 2.1 *El grupo multiplicativo de un cuerpo finito es cíclico.*

A continuación se enuncian algunos teoremas que hacen referencia a las propiedades más importantes de los grupos cíclicos relacionadas con las aplicaciones criptográficas.

Teorema 2.3 $(\mathbb{Z}_p^*, \cdot)$ *es un grupo cíclico finito Abeliano, para todo primo* p.

La propiedad a la que hace referencia el teorema es una generalización del *Pequeño Teorema de Fermat* para todos los grupos cíclicos. En él se establece que el grupo multiplicativo de cada **cuerpo primo** es cíclico. Estos grupos son los más utilizados en la construcción de las primitivas criptográficas basadas en el *logaritmo discreto*.

Teorema 2.4 *Sea* $\mathbb{G}$ *un grupo finito. Entonces para todo* $a \in \mathbb{G}$ *se cumple que:*

1. $a^{|\mathbb{G}|} = 1$.
2. *Ord*(a) *divide a* $|\mathbb{G}|$

Teorema 2.5 *Sea* $\mathbb{G}$ *un grupo cíclico finito. Entonces se cumple que:*

1. *El número de elementos primitivos de* $\mathbb{G}$ *es* $\phi(|\mathbb{G}|)$.
2. *Si* $|\mathbb{G}|$ *es primo, entonces todos los elementos* $a \neq 1 \in \mathbb{G}$ *son primitivos.*

Definición 2.8 *Sea* $\mathbb{G}$ *un grupo y el elemento* $a \in \mathbb{G}$. *El orden de* a, $Ord_{\mathbb{G}}(a)$, *es el menor entero positivo* n *tal que* $a^n = e$. *Si* $a^n = 1$, *entonces* $n = 0$, *y* a *tiene orden infinito. Si* $\mathbb{G}$ *es un grupo finito, no hay elementos de orden infinito y* $a^{|\mathbb{G}|} = 1$ *para todo elemento* a *de* $\mathbb{G}$.

Proposición 2.2 *Sea* a *un elemento de* $\mathbb{G}$ *de orden* r. *Entonces:*

1. *El grupo generado por* a *es* $< a >= 1, a, a^2, ..., a^{r-1}$.
2. *Si* t *es un entero tal que* $a^t = 1$ *se cumple que* $r|t$. *(Se lee* r *divide a* t*)*
3. *Si* $a^n = a^m$ *para ciertos enteros* n, m *se tiene que* $n \equiv m \pmod{r}$.

Proposición 2.3 *Sea* $\mathbb{G}$ *un grupo finito de orden* n. *El orden de cada subgrupo de* $\mathbb{G}$ *y el orden de cada elemento de* $\mathbb{G}$ *es divisor de* n.
Si n *es primo,* $\mathbb{G}$ *es cíclico y no posee subgrupos propios.*
Si $\mathbb{G}$ *es cíclico, para cada divisor positivo* d *de* n, *existirá un único subgrupo de orden* d *de* $\mathbb{G}$.

Teorema 2.6 *Todo grupo abeliano finito* $\mathbb{G} \neq 0$ *es isomorfo (o estructuralmente iguales) a un producto de grupos cíclicos finitos de órdenes* $m_1, \ldots, m_t$ *donde los componentes* m_i *son mayores que 1 y primos entre sí.* $\mathbb{G} \approx \mathbb{Z}/m_1\mathbb{Z} \times ... \times \mathbb{Z}/m_t\mathbb{Z}$

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

Este isomorfismo puede calcularse mediante el *Teorema Chino del resto*:

Teorema 2.7 ***Teorema Chino del resto*** *Sean $m_1, m_2, \ldots, m_r$ enteros positivos dos a dos primos entre sí, y $a_1, a_2, \ldots, a_r$ enteros cualesquiera.*

$$\begin{aligned} x &\equiv a_1 \text{ mód } m_1 \\ x &\equiv a_2 \text{ mód } m_2 \\ &\vdots \quad\quad \vdots \\ x &\equiv a_r \text{ mód } m_r \end{aligned}$$

El sistema de congruencias tiene solución única módulo $m = m_1 \cdot m_2 \cdot, \ldots, \cdot m_r$.

De este teorema se puede deducir que:

1. El grupo $\mathbb{Z}/m\mathbb{Z}$ es un grupo finito con la suma.
2. El conjunto de las unidades de $\mathbb{Z}/m\mathbb{Z}, (\mathbb{Z}/m\mathbb{Z})^*$ es un grupo de cardinalidad $\phi(m)$ donde ϕ es la función de Euler. Por lo tanto, para cada elemento a de $(\mathbb{Z}/m\mathbb{Z})^*, a^{\phi(m)} = 1 \text{ mód } m$. Este resultado es el conocido *Teorema de Euler*.

Veamos un ejemplo donde aplicamos las definiciones y teoremas anteriores:

Ejemplo 2.4 *Se quiere comprobar si $a = 5$, es un elemento primitivo (generador) de $\mathbb{Z}_7^* = 1, 2, 3, 4, 5, 6$. La cardinalidad del grupo es $|\mathbb{Z}_7^*| = 6$, dado que $\phi(7) = 6$. Veamos los elementos que se generan por potencias de $a = 5$:*

$$\begin{aligned} a^1 &\equiv 5 \\ a^2 &\equiv 4 \text{ mód } 7 \\ a^3 &\equiv 6 \text{ mód } 7 \end{aligned} \qquad \begin{aligned} a^4 &\equiv 2 \text{ mód } 7 \\ a^5 &\equiv 3 \text{ mód } 7 \\ a^6 &\equiv 1 \text{ mód } 7 \end{aligned}$$

De este resultado se concluye que:

- *Ord*$(a) = 6 = |\mathbb{Z}_7^*|$.
- $a = 5$ *es un elemento primitivo.*
- $|\mathbb{Z}_7^*|$ *es cíclico.* ■

Verifiquemos que las potencias de $a = 5$ realmente generan todos los elementos del grupo $\mathbb{Z}_7^*$

Cuadro 2.1: Verificación de los elementos del grupo

i	1	2	3	4	5	6
a^i	5	4	6	2	3	1

En el Cuadro 2.1 se comprueba que las potencias de 5^i realmente generan todos los elementos del grupo $\mathbb{Z}_7^*$ como se observa en la fila de abajo, pero el orden es arbitrario. Esta aparente aleatoriedad entre el exponente i y los elementos del grupo constituye la base para protocolos como el "Intercambio de claves Diffie-Hellman".

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

SUBGRUPOS CÍCLICOS

Dado un grupo $\mathbb{G}$ con una operación binaria $\circ$, se dice que un subconjunto no vacío H de $\mathbb{G}$ es un subgrupo de $\mathbb{G}$ si H también forma un grupo bajo la misma operación $\circ$. Existe una manera fácil de generar subgrupos en el caso de grupos cíclicos que se deduce del siguiente teorema.

Teorema 2.8 ***Subgrupo cíclico****: Sea $(\mathbb{G}, \circ)$ un grupo cíclico. Entonces todo elemento $a \in \mathbb{G}$ con Ord$(a) = s$ es el elemento primitivo de un subgrupo cíclico con s elementos.*

Este teorema nos dice que cualquier elemento de un grupo cíclico es el generador de un subgrupo que a su vez también es cíclico. Para comprobar si un subconjunto H de un grupo $\mathbb{G}$ es un subgrupo, se puede verificar si todas las propiedades de la definición de grupo en la Sec. 2.1.2, se cumplen para H.

A continuación veremos dos teoremas que afectan a los *Subgrupos de primer orden*.

Teorema 2.9 ***Teorema de Lagrange****: Sea H un subgrupo de $\mathbb{G}$. Entonces $|H|$ divide a $|\mathbb{G}|$. Luego para $a \in \mathbb{G}$, el orden de a divide a $|\mathbb{G}|$.*

Consecuentemente se establece que el orden de un subgrupo divide al orden del grupo.

Teorema 2.10 *Sea $\mathbb{G}$ un grupo cíclico finito de orden n y sea α un generador de $\mathbb{G}$. Entonces para cada entero k que divide a n existe exactamente un subgrupo cíclico H de $\mathbb{G}$ de orden k. Este subgrupo es generado por $\alpha^{n/k}$. H está compuesto de los elementos $a \in \mathbb{G}$ que satisfacen la condición $a^k = 1$. No existen otros subgrupos.*

Este teorema nos da un método para construir rápidamente un subgrupo a partir de un grupo cíclico dado. Lo único que necesitamos es un elemento primitivo y la cardinalidad del grupo n. Se calcula $\alpha^{n/k}$ y se obtiene un generador del subgrupo con k elementos.

2.1.4. ANILLOS

Definición 2.9 *Un anillo,R, es un conjunto no vacío, dotado con dos operaciones binarias, $(+)$ y $(*)$: suma y multiplicación, de modo que se verifican las siguientes propiedades:*

1. R es un grupo abeliano con respecto a la suma.
2. Para cada $g, h \in R$, el producto $g * h$ está en R.
3. La multiplicación $(*)$ es una operación asociativa en R.
4. La multiplicación $(*)$ es distributiva respecto a la suma.

Si la multiplicación en R es conmutativa, entonces R es un *anillo conmutativo*. Si existe el elemento identidad para la multiplicación, 1 tal que $g * 1 = 1 * g = g$ para todo $g \in R$ se dice que R es un *anillo con unidad*.

Dentro de los anillos unitarios y conmutativos, aquellos que no tienen divisores de cero se les denomina *dominios de integridad*.

Un anillo utilizado con frecuencia es el anillo de enteros, el cuál es conmutativo y contiene infinitos elementos.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

Es importante observar que el conjunto de las clases residuales de enteros módulo n, denotado por $\mathbb{Z}_n$, es un anillo conmutativo con un *número finito* de elementos bajo la suma y la multiplicación reducida módulo n. Este anillo contiene exactamente n elementos enteros $\{0, 1, ..., n-1\}$.

2.1.5. CUERPOS FINITOS DE GALOIS

La teoría de los cuerpos finitos se conoce también como teoría de cuerpos[1] de Galois, en honor a Evariste Galois, matemático francés del siglo XIX. Toda la criptografía moderna está basada en la teoría de los cuerpos finitos de Galois. Por ejemplo, sin entender la noción de cuerpos finitos, es imposible conocer cómo funciona el AES, actual estándar de cifrado en bloque. Los pasos de sustitución que se emplean en el AES están basados en el concepto de *inverso multiplicativo en un cuerpo finito*. Por su parte, el algoritmo RSA y la criptografía de clave pública, así como una serie de protocolos modernos tienen como base matemática toda esta teoría.

En computación ordinaria, la división es particularmente propensa a errores, y lo que observamos como resultado es una aproximación del verdadero resultado con muy alta precisión. Tal altísima precisión no es suficiente en criptografía. En criptografía todas las operaciones aritméticas deben trabajar sin error. Veamos algunas definiciones importantes relacionadas con los cuerpos:

Un cuerpo es una estructura algebraica, en la que se pueden definir dos operaciones de composición de modo único: la suma y la multiplicación. Las operaciones inversas serían la resta y la división respectivamente.

Definición 2.10 *Un cuerpo $\mathbb{F}$ es un anillo conmutativo con identidad con la propiedad adicional de que el conjunto de los elementos distintos de cero de $\mathbb{F}$, forman un grupo Abeliano bajo la multiplicación.*

Por tanto un cuerpo es un conjunto $\mathbb{F}$ en el que se han definido dos operaciones, llamadas adición y multiplicación respectivamente, que cumplen las propiedades asociativa, conmutativa y distributiva de la multiplicación respecto de la adición, además de la existencia del inverso aditivo, del inverso multiplicativo y de un elemento neutro para la adición y otro para la multiplicación, los cuales permiten efectuar las operaciones de sustracción y división (excepto la división por cero).

Definición 2.11 *Un cuerpo finito es un cuerpo con un número finito de elementos. El orden de un cuerpo finito es el número de elementos del mismo y siempre es un número primo o una potencia de un primo.*

El conjunto de los enteros con las operaciones de suma y multiplicación no es un cuerpo ya que sólo tienen inverso 1 y -1, y la mayoría de los elementos carecen de inverso multiplicativo.

Definición 2.12 *La característica de un cuerpo $\mathbb{F}$, $char(\mathbb{F})$, es el menor entero positivo r tal que $r \cdot 1 = 0$. Si la relación $r \cdot 1 = 0$ implica que $r = 0$, entonces $char(\mathbb{F}) = 0$.*

Existe un cuerpo finito de orden q si y solo sí q es potencia de un primo. Si q es potencia de un primo, entonces, existirá solo un cuerpo finito de orden q; este cuerpo se denota por $\mathbb{F}_q$. Como existen muchas maneras de representar los elementos de $\mathbb{F}_q$, algunas representaciones pueden conducir a implementaciones más/menos eficientes de la aritmética del cuerpo en hardware o software.

[1] En inglés, a un cuerpo se le llama *field* cuya traducción literal castellana es "campo", en algunas traducciones al español aparece así escrito, aunque es incorrecto.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

Definición 2.13 *Si $\mathbb{F}_q$ es un cuerpo finito de orden $q = p^m$, siendo p un número primo, entonces la* característica *de $\mathbb{F}_q$ es p.*

Teorema 2.11 *El anillo de las clases residuales $\mathbb{Z}_p$ módulo el ideal principal generado por p es un cuerpo sí y sólo sí p es un número primo. Este cuerpo se denomina cuerpo primo o cuerpo de Galois de p elementos $GF(p)$ y su característica es p.*

Para cualquier entero a, la operación $a\,(mod\,p)$ indica que el único residuo entero r, obtenido dividiendo a por p está en el intervalo $0 \leq r \leq p-1$, a esta operación se le denomina *reducción módulo p*.

Un cuerpo finito de q elementos se representa por $GF(q)$, los cuerpos de extensión $GF(2)$ representados por $GF(2^n)$ se utilizan en muchas aplicaciones.

Proposición 2.4 *La característica de un cuerpo es siempre cero o un número primo.*

Proposición 2.5 *Sea $\mathbb{F}$ un cuerpo finito. Entonces $char(\mathbb{F})$ es un primo p y $|\mathbb{F}| = p^m$ para algún entero $m \geq 1$. Por lo tanto, si $|\mathbb{F}| = p^m$, el orden de $\mathbb{F}^*$ es, $\phi(p^m) = (p-1)p^{m-1}$.*

Como ya hemos mencionado la aritmética sobre un cuerpo finito de Galois se puede llevar a cabo exactamente con ordenadores, por ello son de suma importancia en las comunicaciones digitales modernas. En criptografía es una herramienta indispensable: Muchas de las operaciones criptográficas se realizan a nivel de byte, utilizando el cuerpo de Galois $GF(2^8)$ se realiza el cifrado y descifrado en el mismo cuerpo, esto elimina los problemas de redondeo y/o truncamiento propios de la aritmética real. Además se utiliza en la generación de números aleatorios, herramienta básica para el diseño de protocolos y sistemas criptográficos, marcas de agua, entre otros. Por otra parte, la aritmética sobre el cuerpo de Galois $GF(2^8)$ se utiliza en los códigos correctores de errores Reed-Sólomon de suma importancia en las comunicaciones por satélite, las redes de ordenadores y los sistemas de almacenamiento óptico y magnético. Un libro que recomendamos para ver la utilidad de los cuerpos finitos en criptografía con implementaciones en Java, aunque es del 2003, pueden encontrarlo aquí.

2.1.6. EL PROBLEMA DEL LOGARITMO DISCRETO

Toda la teoría matemática de la Sec. 2.1 anterior, está relacionada directamente con el protocolo de intercambio de claves de Diffie y Hellman (DHKE[2]), pues tanto la función unidireccional del DHKE, como el problema del logaritmo discreto (PLD), se explican a través de la teoría de grupos cíclicos. Volvamos a la definición del problema:

El Problema del Logaritmo Discreto en Z_p^*: Dados el grupo cíclico finito Z_p^* de orden $p-1$, un elemento primitivo $\alpha \in Z_p^*$ y otro elemento $\beta \in Z_p^*$. El problema del logaritmo discreto consiste en determinar el entero $1 \leq x \leq p-1$, tal que:

$$\alpha^x \equiv \beta \text{ mód } p$$

[2]Diffie Hellman Key Exchange

El entero x existe ya que α es un elemento primitivo y cada elemento del grupo puede ser expresado como una potencia de cualquier elemento primitivo. El entero x se denomina *el logaritmo discreto* de β en base a α, y puede escribirse de la forma:

$$x = \log_\alpha \beta \bmod p \tag{2.1}$$

Esta es una función unidireccional, ya que el cálculo del logaritmo discreto módulo un primo, Ec. (2.1), es un problema intratable, si los parámetros son suficientemente grandes. Por otra parte la función inversa, es decir la exponenciación,

$$\alpha^x \equiv \beta \bmod p, \tag{2.2}$$

es fácil de calcular computacionalmente. La importancia del estudio de este problema radica en el interés del logaritmo discreto como operación inversa a la exponenciación en un grupo.

Como hemos visto, la exponenciación modular es una operación sencilla y se conocen métodos eficientes de calcularla. En cambio, el cálculo del logaritmo discreto módulo un entero cualquiera no siempre puede realizarse.

El problema del logaritmo discreto (PLD) consiste entonces en calcular x, conocidos α, β.

Ejemplo 2.5 *Si consideramos el logaritmo discretoen el grupo $\mathbb{Z}_{47}^*$ en el que $\alpha = 5$ es un elemento primitivo. Para $\beta = 41$ el problema del logaritmo discreto será encontrar el entero x tal que $5^x \equiv 41 \bmod 47$. Para estos número pequeños, determinar x no es directo. Usando un ataque por fuerza bruta, es decir probando con todos los posibles valores de x, se obtiene que $x = 5$.* ■

En la práctica para evitar ataques como el de Pohlig-Hellman se trabaja con subgrupos de $\mathbb{Z}_p^*$ con orden primo.

Método binario de exponenciación modular: Si deseamos calcular b^e en el grupo multiplicativo de los enteros módulo m. En primer lugar, hay que escribir el exponente en binario:

$e = \sum_{i=0}^{n-1} a_i 2^i$

A continuación:

1. Se definen $i = n - 1$, y $v = 1$
2. Para $i = n - 1$ hasta 0,
3. Se calcula: $f(x) = \begin{cases} v = vb \pmod m & \text{si } a_i = 1 \\ v = v & \text{si } a_i \neq 1 \end{cases}$
4. Luego, $f(x) = \begin{cases} v = v & \text{si } i = 0 \\ v = v^2 \pmod m & \text{si } i \neq 1 \end{cases}$

Tras la última iteración se tiene $v = b$.

Ejemplo 2.6 *Sea $\mathbb{Z}_5^*$ el grupo multiplicativo de los enteros módulo 5. Se quiere calcular 3^{27}. Esto puede hacerse de cualquiera de las formas explicadas:*

- *Realizando las 26 multiplicaciones:* $3^{27} \equiv 2 \pmod 5$.
- *Utilizando el método binario de exponenciación modular. Como* $27 = 2^4 + 2^3 + 2 + 1$, *la representación en binario de* 27 *es* 11011. *Entonces, se toma* $i = 4$, $v = 1$ *y siguiendo los pasos indicados construimos la tabla 2.2.*

Cuadro 2.2: Cálculo de 3^{27} en $\mathbb{Z}_5^*$.

i	a_1	v
4	1	$(1 \cdot 3)^2 \equiv 4 \pmod 5$
3	1	$(4 \cdot 3)^2 \equiv 4 \pmod 5$
2	0	$4^2 \equiv 1 \pmod 5$
1	1	$(1 \cdot 3)^2 \equiv 4 \pmod 5$
0	1	$(4 \cdot 3) \equiv 2 \pmod 5$

Por lo tanto, la solución es: $3^{27} \equiv 2 \mod 5$■

Con este método, el número necesario de multiplicaciones pasa de 26 a 8. Esto evidencia que la exponenciación podría ser una buena función trampa. Es por ello que algunos sistemas de cifrado basan su seguridad en el problema del logaritmo discreto.

2.2. INTERCAMBIO DE CLAVES DE DIFFIE-HELLMAN

Como ya hemos explicado, los criptosistemas convencionales que utilizan algoritmos de clave secreta presentan el problema de la gestión de claves: por un lado, la distribución de claves y por otra parte, el problema de la gestión de claves cuando el número de usuarios involucrados en la comunicación es grande, ya que en una red de n usuarios, cada pareja necesita tener su clave secreta particular, lo que hace un total de $2^{n(n-1)}$ claves para esa red de usuarios.

En 1976, Whitfield Diffie y Martin Hellman propusieron el primer Protocolo de Intercambio de claves (DHKE) basado en la exponenciación en grupos finitos [1]. El protocolo permite que dos usuarios intercambien pequeñas cantidades de información de forma segura, usando un canal inseguro.

Su aporte fundamental es que proporciona una solución práctica al problema de distribución de claves, es decir, permite a las dos partes que intervienen en la comunicación derivar una clave secreta común utilizando un canal inseguro (que tendrá que ser autentificado). Así, el DHKE es un protocolo que sirve para el establecimiento de claves secretas entre partes que no han tenido contacto previo y que comparten un canal inseguro de manera anónima, utilizando solo dos mensajes para acordar dicha clave. La clave secreta resultante no puede ser descubierta por un atacante aunque éste obtenga los dos mensajes enviados durante el protocolo.

La seguridad del DHKE radica precisamente en la extrema dificultad (conjeturada, no demostrada) de calcular logaritmos discretos en un cuerpo finito. Este protocolo se utiliza ampliamente en la implementación de muchos protocolos criptográficos abiertos y comerciales, como por ejemplo, el Secure Shell (SSH), en la Seguridad de la Capa de Transporte (TLS) y Seguridad del Protocolo de Internet (IPSec).

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

2.2.1. FUNCIONAMIENTO DEL INTERCAMBIO DE CLAVES DE DIFFIE-HELLMAN EN $\mathbb{Z}_P^*$

Alice y Bob representan las dos partes que quieren establecer una clave secreta conjunta. Puede existir una entidad que actúe como tercera parte de confianza y se encargue de elegir los parámetros que son necesarios para el intercambio de claves. Sin embargo, también es posible que Alice y Bob generen por si mismos los parámetros públicos [3].

Estrictamente hablando, el protocolo DHKE se divide en dos fases: la fase de configuración, donde se establecen los parámetros de dominio y el protocolo principal (Intercambio de Claves).

DHKE: CONFIGURACIÓN DE PARÁMETROS DE DOMINIO

Supongamos que Alice (A) y Bob (B) quieren acordar una clave secreta común usando un canal inseguro. Los dos usuarios A y B, seleccionan públicamente un grupo cíclico multiplicativo finito $\mathbb{G}$ (es decir, la operación interna es el producto), cuyo orden es $|G| = n$, y un generador del mismo, α.

Luego eligen un número aleatorio secreto $a, (1 \leq a \leq p-1)$ y $b, (1 \leq b \leq p-1)$ respectivamente y comienzan el protocolo dado en la Fig. 8 cuyo resultado es la clave secreta k_{AB} a utilizar en la comunicación,

Parámetros de dominio

1. Se eligen un número primo p, y un generador $\alpha \in \mathbb{G}_p^*$ del grupo cíclico finito $\mathbb{G}_p^*(2 \leq \alpha \leq p-2)$.
2. Hacer públicos p, α.

Una vez que Alice y Bob eligen los parámetros de dominio p y α en la fase de configuración, pueden entonces generar una clave secreta conjunta k_{AB} con el siguiente protocolo de intercambio de claves.

DHKE: PROTOCOLO

Después realizar el protocolo, A y B comparten un elemento común del grupo $\mathbb{G}$ que es secreto. Dicho elemento es el siguiente: $k = (\alpha^b)^a = (\alpha^a)^b = \alpha^{ab}$.

Un intruso o criptoanalista C que intervenga la transmisión realizada por Alice y Bob, podía conocer $\mathbb{G}, n, \alpha, \alpha^a$ y α^b. El problema que C debe resolver sería el de calcular el elemento α^{ab}. Este problema es conocido como el Problema de Diffie-Hellman. Este es un problema muy difícil de resolver desde el punto de vista computacional ya que actualmente no es posible obtener su solución en un tiempo menor que algunos miles de años, si los parámetros del protocolo se elijen de forma adecuada. Para resolver este problema de forma práctica es necesario encontrar un algoritmo que sea eficiente computacionalmente.

El Problema del Logaritmo Discreto (Sec.2) consiste en encontrar un algoritmo eficiente computacionalmente que calcule el valor de a, conocidos $\mathbb{G}, n, \alpha, \alpha^a$. Resulta claro que si el intruso C fuera capaz de resolver el *Problema del Logaritmo Discreto*, habría resuelto también *el Problema de Diffie-Hellman*. Generalmente se acepta la conjetura que ambos problemas son equivalentes, por lo que si se resuelve uno de ellos se puede fácilmente resolver el otro en tiempo polinómico. De ahí la afirmación que la seguridad del protocolo de intercambio de clave de Diffie-Hellman está basada en la seguridad del logaritmo discreto.

Protocolo de Intercambio de claves Diffie y Hellman

Alice		**Bob**
	Generan aleatoriamente en secreto:	
$a = k_{prA} \in \{2, \ldots, p-2\}$		$b = k_{prB} \in \{2, \ldots, p-2\}$
Calcula: $k_{pubA} = \alpha^a \bmod p$		Calcula: $k_{prB} = \alpha^b \bmod p$
	A envía: $\alpha^a \bmod p \longrightarrow$	
	$\longleftarrow$ B envía: $\alpha^b \bmod p$	
Alice calcula:		Bob calcula:
$k = (\alpha^b)^a \bmod p$		$k = (\alpha^a)^b \bmod p$

Figura 3: Protocolo de Intercambio de claves de Diffie y Hellman.

Observación 2.2 *Es muy importante saber que el protocolo de intercambio de clave de Diffie-Hellman no es un criptosistema ya que no realiza las funciones de cifrado y descifrado de la información, sólo permite el intercambio de pequeñas cantidades de información, por medio de un canal inseguro.*

Ejemplo 2.7 *Alice y Bob eligen como parámetros de dominio para comenzar el DHKE,* $p = 29$ *y* $\alpha = 2$*. Para acordar la clave siguen los pasos mostrados en la Fig. 4. Como se observa, ambas partes calculan el valor* $k_{AB} = 16$*, que puede ser utilizado como una clave de sesión para un cifrado simétrico.*■

Ejemplo: Intercambio de claves Diffie y Hellman

Alice		**Bob**
	Generan aleatoriamente en secreto:	
$a = k_{prA} = 5$		$b = k_{prB} = 12$
Calcula: $k_{pubA} = 2^5 \bmod 29$		Calcula: $k_{prB} = 2^{12} \bmod 29$
	A envía: $k_{prA} = 3 \longrightarrow$	
	$\longleftarrow$ B envía: $k_{prB} = 7$	
Alice calcula:		Bob calcula:
$k_{AB} = 7^5 \equiv 16 \bmod 29$		$k_{AB} = 3^{12} \equiv 16 \bmod 29$

Figura 4: Ejemplo numérico del Intercambio de claves de Diffie y Hellman.

2.2.2. SEGURIDAD DEL INTERCAMBIO DE CLAVES DIFFIE-HELLMAN

El protocolo de intercambio de claves de Diffie-Hellman es seguro contra un espía pasivo, pero es vulnerable frente a ataques de hombre en medio (*man-in-the-middle attack* MitM): un atacante e podría situarse entre ambas máquinas y acordar una clave simétrica con cada una de las partes, haciéndose pasar por A de cara a B y viceversa. Una vez establecidas las 2 claves simétricas, el atacante haría de puente entre los 2 participantes en la comunicación, A y B, descifrando toda la comunicación y volviéndola a cifrar para enviársela al otro interlocutor.

Este protocolo es también vulnerable si un atacante logra engañar a los usuarios para que usen parámetros sesgados (p, α) por ejemplo, que utilicen un valor de α que solo genere un pequeño subgrupo. El atacante podrá hacer una búsqueda por fuerza bruta en este conjunto mucho más pequeño.

Consideremos la posibilidad de un intruso pasivo, que pueda escuchar en la comunicación pero no alterar los mensajes. Su objetivo será calcular la clave de k_{AB} compartida entre Alice y Bob. El intruso conoce α y p, pues estos parámetros son públicos durante la fase de configuración del protocolo. También podrá obtener los valores $A = k_{pub,A}$ y $B = k_{pub,B}$ "escuchando" en el canal durante una ejecución del protocolo de intercambio de claves. ¿Será capaz de calcular $k = \alpha^{ab}$ a partir de $\alpha, p, A \equiv \alpha^a \mod p$ y $B \equiv \alpha^b \mod p$. La incógnita se conoce como el Problema de Diffie y Hellman (PDH) y puede ser generalizado a grupos cíclicos finitos arbitrarios.

> Es importante tener en cuenta que hasta la fecha no se conoce si resolver el Problema del Logaritmo Discreto (PLD) es la única manera de solucionar el Problema de Diffie-Hellman. Es por esto que para garantizar la seguridad de la DHKE en la práctica, tenemos que comprobar que el PDL correspondiente no puede resolverse.

Esto se logra eligiendo p suficientemente grande como para que no pueda ser calculado. Un requisito adicional es que el factor primo más pequeño de $p - 1$ (orden del grupo cíclico) debe ser por lo menos de longitud 160 bits para un nivel de seguridad de 80 bits, y al menos de 256 bits de longitud para un nivel de seguridad de 128 bits.

MEJORAS EN LA IMPLEMENTACIÓN DEL PROTOCOLO DHKE

1. Protocolo Diffie-Hellman usando un subgrupo cíclico grande: Actualmente existen algunos algoritmos que mejoran el protocolo DHKE: Para evitar ataques de "grupos pequeños", nos aseguramos de que el orden $p - 1$ del grupo tenga un divisor primo grande q y que α sea un generador del subgrupo cíclico único de orden $q \mod p$. El método mejorado del DHKE usa los parámetros del dominio (p, q, α). Estos parámetros son utilizados algoritmos por todos los derivados el DHKE. Este método se describe en ANSI X9.42 y RFC 2631. La técnica de usar un orden principal se debe a Claus-Peter Schnorr [10].

2. Acuerdo de clave MQV: Es una mejora del protocolo Diffie-Hellman original diseñada para dificultar el ataque MitM. Lleva el nombre de Menezes, Qu y Vanstone. Aunque el documento original está escrito para la criptografía de curva elíptica, el protocolo también funciona con logaritmos discretos. MQV supone que Alice y Bob, ya han establecido y autenticado sus pares de claves estáticas (a, A) y (b, B) entre sí; es decir, Alice confía en que la clave pública B es realmente de Bob, y Bob confía en la clave pública A de Alice. Cada parte genera un par de *claves temporales o efímeras* al azar $(x, X = gx)$ y $(y, Y = gy)$, y estos se usan junto con las claves estáticas para crear un nuevo secreto compartido. Para profundizar ver: [11]

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

2.2.3. EJEMPLO DE USO DEL PROTOCOLO DHKE

El intercambio de claves de Diffie y Hellman es ampliamente utilizado en innumerables aplicaciones del mundo real, entre ellas nombraremos el protocolo *Transport Layer Security* (TLS)[a] que incluye soporte para el intercambio de claves Diffie-Hellman de clave efímera. El DHKE proporciona el secreto hacia adelante perfecto (*forward secrecy*).

[a]Será estudiado en el módulo 2: Seguridad web.

En el protocolo, el servidor especifica su exponente, el módulo primo y el generador. Para evitar puertas traseras, el cliente debe asegurarse de que el módulo y el generador sean seguros. El ataque MitM se evita al requerir que el exponente del DHKE del servidor sea autenticado. Los clientes anónimos no están obligados a tener un certificado.

En RFC 4492 se describen los algoritmos del protocolo de intercambio de claves basados en criptografía con curva elíptica (ECC) para la seguridad del protocolo TLS. En particular, se especifica el uso del acuerdo de clave de Diffie-Hellman de curva elíptica (ECDH) en un protocolo de enlace TLS y el uso del algoritmo de firma digital de curva elíptica (ECDSA) como un nuevo mecanismo de autenticación. A continuación se resumen los algoritmos de intercambio de claves y autenticación en TLS 1.2.

Código	Algoritmos	Observaciones
TLS-RSA	RSA	RSA estático, obsoleto.
TLS-DH-RSA	Diffie-Hellman con RSA	
TLS-DHE-RSA	Diffie-Hellman Efímero con RSA	Secreto hacia adelante
TLS-ECDH	Diffie-Hellman con curva elíptica	
TLS-ECDHE	DH con curva elíptica Efímero	Secreto hacia adelante
TLS-DHE-RSA	Diffie-Hellman Efímero con RSA	Secreto hacia adelante
TLS-DH-anon	DH anónimo	Sin autenticación
TLS-PSK	Con clave pre-compartida	La clave se comparte usando un canal seguro.

Cuadro 2.3: Algoritmos de intercambio de claves y autenticación en TLS 1.2.

En los sistemas con seguridad hacia adelante (*Perfect Forward Security*), las claves públicas utilizadas para intercambiar claves privadas solo se usan una vez por sesión, y no se generan por ningún algoritmo determinista (predecible).

A día de hoy (febrero2018) TLS1.3 es un borrador sobre el que se está trabajando. Para seguir su desarrollo ver: TLS working group. La guía de configuración e implementación de este protocolo se encuentra en, NIST SP-800-52r1.

2.3. CRIPTOSISTEMA DE ELGAMAL

El criptosistema ElGamal [6], es un algoritmo de cifrado de clave pública basado en el problema del logaritmo discreto discreto. Fue descrito por Taher ElGamal en 1985. Puede ser utilizado tanto para generar firmas digitales como para cifrar o descifrar. Aunque generalmente no se utiliza de forma directa, ya que la velocidad de cifrado y autenticación es inferior a la obtenida con RSA, y además las firmas producidas son más largas, el algoritmo de ElGamal es de gran importancia en

el desarrollo del DSS (Digital Signature Standard), del NIST (*National Institute of Standards and Technology*) de los Estados Unidos. El criptosistema ElGamal tiene una característica determinante que lo distingue del resto de sistemas de clave pública: en el cifrado se utiliza además de la clave pública del receptor, la clave privada del emisor. Desde su publicación, se han propuesto diferentes criptosistemas alternativos, basados en la propuesta original tal y como se describe en [12].

La seguridad del criptosistema se basa en el problema del logaritmo discreto. El mejor algoritmo conocido para resolverlo es la "criba general de cuerpo numérico", cuyo tiempo de ejecución es subexponencial. A continuación examinaremos las tres fases del criptosistema ElGamal: generación de claves, cifrado de mensajes y descifrado de mensajes.

2.3.1. FASE DE GENERACIÓN DE CLAVES

En primer lugar, el usuario que va a comunicarse de manera segura utilizando este criptosistema (supongamos que es Alice) debe generar sus claves siguiendo el Protocolo 2.1.

Protocolo 2.1 Protocolo de generación de claves ElGamal

1. Generar un número primo grande p (de 1024 bits).
2. Elegir un generador, α, del grupo $\mathbb{Z}_p^* = \{\alpha^i : i = 1, \ldots, p-1\}$.
3. Generar un número aleatorio a, con $1 < a < p-2$, y calcula $\alpha^a \bmod p$.
4. La clave pública de Alice será (p, α, α^a) y su clave privada es a.

La longitud mínima recomendada de la clave pública p es de 1024 bits, la misma que la recomendada para el RSA, aunque en ElGamal p es un primo, mientras que en el RSA n es el producto de dos primos.

2.3.2. FASE DE CIFRADO

Si Bob quiere cifrar un mensaje M y enviarlo a Alice seguirá el Protocolo 2.2.

Protocolo 2.2 Protocolo de cifrado ElGamal

1. Obtener la clave pública de Alice: $(p, \alpha, A = \alpha^a)$.
2. Representar el mensaje M como un elemento de $\mathbb{Z}_p^*$: se representa el mensaje como un entero $m \in [1, p-1]$.
3. Generar un número aleatorio x, $1 < x < p-2$, y calcular

$$f \equiv \alpha^x \bmod p, \quad g \equiv m \cdot A^x \bmod p$$

enviar el texto cifrado obtenido a Alice: $c = (f, g)$.

2.3.3. FASE DE DESCIFRADO

Al recibir el criptograma $c = (f, g)$, para obtener el mensaje original, m, Alice sigue el Protocolo 2.3.

Protocolo 2.3 Protocolo de descifrado ElGamal

1. Utilizar su clave privada, a, para calcular

$$k \equiv f^{p-1-a} \text{ mód } p \left(\equiv \alpha^{x(p-1)}\alpha^{-xa} \text{ mód } p \equiv \alpha^{-xa} \text{ mód } p\right).$$

2. El mensaje descifrado m se obtiene multiplicando el valor anterior por g, es decir:

$$k \cdot g \text{ mód } p \left(\equiv \alpha^{-xa} \cdot m \cdot (\alpha^a)^x \text{ mód } p\right) \equiv m.$$

2.3.4. SEGURIDAD

El criptoanalista conoce las claves públicas de Alice y de Bob, por tanto puede conocer n, α, $A = \alpha^a$ y $B = \alpha^b$. Además si ha tenido acceso a la comunicación conocerá f y g, es decir,

$$f \equiv \alpha^x \quad \text{mód } p, \quad g \equiv m \cdot \alpha^{ax} \text{ mód } p.$$

Para romper el criptosistema, es decir, para determinar cualquiera de las claves privadas de los usuarios A o B, (o determinar el valor de x), tendría que resolver el problema del logaritmo discreto, lo cual es computacionalmente intratable a día de hoy (año 2018).

2.4. ALGORITMO DE FIRMA DIGITAL (DSA)

DSA [13], es una variante de los esquemas de ElGamal y Schnnor, propuesto en 1991 y se convirtió en un estándar del Gobierno Federal de los Estados Unidos de América para firmas digitales (FIPS-180). Se basa en subgrupos pequeños de $\mathbb{Z}_p^*$ con el objetivo de disminuir el tamaño de la firma. Su seguridad está basada en la dificultad de resolver el problema del logaritmo discreto. Se recomienda profundizar este tema consultando los materiales del curso para este tema.

2.5. RESUMEN

En esta sección se han revisado algunas definiciones, teoremas y proposiciones del álgebra abstracta relacionadas con grupos, subgrupos, grupos finitos, grupos cíclicos y cuerpos, que son esenciales para la comprensión de los algoritmos de clave pública basados en el logaritmo discreto.

La importancia del estudio de este problema radica en la utilización del logaritmo discreto como operación inversa a la exponenciación en un grupo: es fácil calcular la exponenciación modular, pero la operación inversa, o sea, el cálculo del logaritmo discreto módulo un entero cualquiera, no siempre puede realizarse si los parámetros son suficientemente grandes (para el caso del protocolo, el número primo debe ser de longitud mayor que 1024), por lo que se puede decir que esta es una función unidireccional. Este sistema es muy utilizado en la actualidad, el criptosistemas y firmas digitales, como los de ElGamal y el DSA, están basados en él. Además el protocolo DHKE es una aplicación del problema del logaritmo discreto y su seguridad radica precisamente en la extrema dificultad de calcular logaritmos discretos en un cuerpo finito. Su limitación radica en que no resuelve el problema de la autentificación, por lo que debe ser utilizado de conjunto con un sistema o algoritmo de firma digital. Se recomienda profundizar en estos algoritmos revisando la bibliografía citada y la bibliografía auxiliar.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

3. CRIPTOSISTEMA BASADO EN LA FACTORIZACIÓN DE ENTEROS: EL RSA

La criptografía de clave pública o asimétrica se caracteriza por que cada usuario utiliza dos claves, una clave privada solo conocida por su dueño y otra pública conocida por todos. Ambas claves están estrechamente relacionadas por un problema matemático [12, 14], de tal modo que obtener la clave privada a partir de la clave pública supone resolver un problema matemático computacionalmente difícil.

Si Bob, desea enviar a Alice un mensaje cifrado mediante un criptosistema de clave pública, debe seguir el siguiente protocolo: Bob localiza la clave pública de Alice, cifra el mensaje haciendo uso de la clave pública de Alice y se lo envía. Por su parte, Alice recibe el mensaje cifrado, lo descifra usando su clave privada, sólo conocida por ella. El sistema basa su fortaleza y seguridad en que sólo el Alice conoce su clave privada, por lo que es la única que puede descifrar los mensajes cifrados con su clave pública.

3.1. TEORÍA DE NÚMEROS

En esta sección presentamos las bases matemáticas elementales para poder comprender diferentes aspectos de los criptosistemas clave pública basados en el problema de la factorización de enteros cuyo máximo exponente es el RSA. Primeramente se describen algunos conceptos básicos necesarios, luego se tratan el Algoritmo Euclidiano, la Función ϕ de Euler, así como el pequeño Teorema de Fermat y el Teorema de Euler. Para profundizar más en estas bases matemáticas consultar Teoría del RSA.

3.1.1. TEORÍA DE LA DIVISIBILIDAD Y NÚMEROS PRIMOS

Definición 3.1 *Sean $a, b \in \mathbb{Z}$ dos números enteros. Se dice que $a|b$ (se lee, a divide a b) si existe otro entero c tal que, $b = a \cdot c$. Siendo a un* divisor *o un* factor *de b; o se puede decir que b un* múltiplo *de a. Si a no divide a b, se escribe $a \nmid b$.*

La operación de dividir un número entero a entre otro entero positivo, $b > 0$, da como resultado un *cociente* c y un *resto* r, tal que

$$a = c \cdot b + r,$$

siendo c y r únicos y $0 \leq r < b$.

Un resultado fundamental de la teoría de la divisibilidad es el teorema de Euclides.

Teorema 3.1 (Teorema de Euclides) *Si un número entero divide al producto de dos enteros, entonces divide al menos a uno de ellos.*

La primera prueba indiscutible del conocimiento de los números primos se remonta a alrededor del año 300 a. C. y se encuentra en el tratado matemático y geométrico que se compone de trece libros denominado los "Elementos de Euclides (tomos VII a IX)". Euclides define los números primos, demuestra que hay infinitos de ellos, define el máximo común divisor (mcd) y el mínimo común múltiplo (mcm) y proporciona un método para determinarlos que hoy en día se conoce como el

algoritmo de Euclides. Los Elementos contienen también el teorema fundamental de la aritmética y la manera de construir un número perfecto a partir de un *número primo de Mersenne*[3]

Se denomina número primo a cualquier entero mayor que 1 divisible únicamente por él mismo y por la unidad: estos son sus únicos factores. Ejemplos de números primos son 2, 3 o 5, aunque en criptografía (sobre todo en la de clave pública) son únicamente útiles los números primos extremadamente grandes, de 512 bits o más, el conjunto de números primos es obviamente infinito.

Definición 3.2 (Número primo) *Un número entero p es* primo *si sólo es divisible por sí mismo y por 1, es decir, si no tiene divisores propios. Si el número entero no es primo se denomina* compuesto.

Teorema 3.2 (Teorema fundamental de la Aritmética) *Todo número compuesto $n \geq 2$ admite una factorización única como producto de potencias de primos:*

$$n = \prod_{i=1}^{k} p_i^{e_i} = p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}$$

donde los p_i son primos tales que $p_1 < \ldots < p_k$ y los e_i son números enteros positivos.

Definición 3.3 (Coprimos) *Dos números enteros a y b se dicen* primos entre sí *o* coprimos *si verifican que el máximo común divisor,* $\mathrm{mcd}(a, b) = 1$.

Definición 3.4 *Dados dos números enteros, a y b, se llama* máximo común divisor *de a y b, dado por* $\mathrm{mcd}(a, b)$*, al mayor número entero que es divisor de a y de b. Se denomina* mínimo común múltiplo *de a y b, dado por* $\mathrm{mcm}(a, b)$*, al menor número entero que es múltiplo de a y de b.*

Para números pequeños, el mcd es fácil de calcular factorizando ambos números y encontrando el factor común más grande.

3.1.2. PRIMALIDAD

El problema de la primalidad consiste en determinar aleatoriamente, números primos grandes (para abundar véase [15]). El problema surge, entre otras razones, porque el criptosistema RSA necesita, para ser implementado, dos números primos, p y q , de longitud grande (de alrededor de 200 dígitos o más). En general, para resolver este problema se recurre a las pruebas de primalidad y de pseudoprimalidad.

[3] En memoria del filósofo del siglo XVII Marin Mersenne quien en su Cognitata Physico-Mathematica realizó una serie de postulados sobre ellos que fueron refinados tres siglos después. También compiló una lista de números primos de Mersenne con exponentes menores o iguales a 257, y conjeturó que eran los únicos números primos de esa forma. Su lista resultó parcialmente correcta y su conjetura falsa, ya que se han descubierto muchos números primos de Mersenne más grandes, el último primo de Mersene, hallado en Diciembre 2017, es $2^{77,232,917} - 1$ con $23,249,425$ dígitos. La verificación rigurosa de su lista, se completó más de dos siglos después.

Definición 3.5 (Primalidad) *Una prueba de primalidad es un criterio para decidir si un número dado es o no primo.*

El tiempo de cálculo necesario para realizar estas pruebas en general es demasiado elevado para llevarlo a la práctica, por lo que generalmente se recurre a las llamadas pruebas de pseudoprimalidad.

Existen muchas pruebas de pseudoprimalidad, es decir probabilísticas, la más sencilla de ellas se basa en el pequeño teorema de Fermat, pero existen otras como la de Sólovay-Strassen [16] y la de Miller-Rabin [16] — que es la más utilizada—.

Pero en el 2004 se ha publicado una prueba de primalidad determinista, ideada por Agrawal, Kayal y Saxena [17], que se basa en una generalización del pequeño teorema de Fermat extendida a los polinomios y en la hipótesis generalizada de Riemann, no demostrada, es por hoy la única que corre en tiempo polinómico.

3.1.3. ALGORITMO DE EUCLIDES

El cálculo del mcd para los números grandes, como los que se utilizan en los sistemas de clave pública, es más complejo, por lo que se utiliza un algoritmo más eficiente denominado Algoritmo de Euclides, que está basado en el teorema de la división de Euclides.

Teorema 3.3 (Teorema de la división de Euclides) *Dados dos números enteros $a > b > 0$, se verifica:*

$$\mathrm{mcd}(a, b) = \mathrm{mcd}(b, r)$$

donde r es el resto de la división de a entre b, es decir, $a = b \cdot c + r$, con $b > r$.

El algoritmo de Euclides sería el siguiente,

Protocolo 3.1 Algoritmo de Euclides

```
1: procedure EUCLIDES(a, b)                    ▷ El máximo común divisor de a, b
2:     r ← a mód b
3:     while r ≠ 0 do                          ▷ Obtendremos la respuesta si r es 0
4:         a ← b
5:         b ← r
6:         r ← a mód b
7:     return b                                ▷ El máximo común divisor es b
```

Como vemos para obtener el $\mathrm{mcd}(a, b)$ de dos enteros $a > b > 0$, se realizan divisiones sucesivas tomando como dividendo y como divisor en cada división el divisor y el resto, respectivamente, de la división anterior, hasta que el resto sea cero. El último divisor es el máximo común divisor.

El *Algoritmo de Euclides* (Algoritmo 3.1) necesita ejecutar $\mathcal{O}\left((\lg a)^3\right)$ operaciones bit; es un algoritmo de tiempo polinómico y, por ende, eficiente. Sin embargo, se puede disminuir aún más el tiempo de ejecución, usando un algoritmo más preciso ([18, §4.3] cuyo tiempo de ejecución quedaría: $\mathcal{O}(\lg a \cdot \lg b)$

3.1.4. ALGORITMO EXTENDIDO DE EUCLIDES

El algoritmo de Euclides no sólo sirve para encontrar el máximo común divisor de dos números, sino que también nos indica que el mcd se puede expresar como combinación lineal de los mismos.

Además, el mcd es el menor entero positivo en que estos dos números se pueden usar como combinación lineal. El algoritmo extendido de Euclides, nos da los coeficientes de esta combinación lineal cuya existencia la garantiza la igualdad de Bezout 3.1.

Proposición 3.1 (igualdad de Bezout) *Si $d = \mathrm{mcd}(a, b)$, existen dos números enteros, u, v, tales que: $d = u \cdot a + v \cdot b$.*

Esto resulta de gran utilidad, sobre todo al momento de resolver ecuaciones Diofantinas (ecuaciones de la forma $au + bv = d$, donde todos son enteros y a, b y d son constantes). Otra aplicación útil es que cuando a y b son coprimos ($d = 1$), entonces u es el inverso multiplicativo de $a \bmod b$.

El algoritmo extendido de Euclides tiene como entrada dos enteros $a > b > 0$ y proporciona como salida los números enteros u, v y d tales que $d = \mathrm{mcd}(a, b) = u \cdot a + v \cdot b$. Utilizando el algoritmo de Euclides 3.1, se va despejando desde la última división obtenida hasta llegar a la primera, para obtener a d como función de a y de b. El código en C del algoritmo de acuerdo a [12] puede descargarse en Euclides.

3.1.5. FUNCIÓN ϕ DE EULER

Otra herramienta útil en los sistemas de clave pública, especialmente para RSA, es la función ϕ de Euler. Recordemos que dos números enteros a y b son relativamente primos si $\mathrm{mcd}(a, b) = 1$. Por ejemplo, 3 y 16 son primos relativos, pero 7 y 28 no. Consideremos $\mathbb{Z}_n$ como el anillo de enteros $\{0, 1, ..., n-1\}$. Nos interesa conocer cuántos números son relativamente primos a n. Esta cantidad viene dada por la función ϕ de Euler, que se define como sigue:

Definición 3.6 (Indicador de Euler) *Sea $n \geq 1$ un entero. Se denota por $\phi(n)$ el número de enteros en el intervalo $[1, n]$ que son primos con n. La función $n \mapsto \phi(n)$ se denomina* indicador ϕ de Euler.

Por definición de indicador de Euler se tiene que $\phi(n) = |\mathbb{Z}_n^*|$. Algunas de las propiedades de este indicador son las siguientes:

1. Si p es un primo, entonces $\phi(p) = p - 1$.
2. El indicador ϕ de Euler es multiplicativo, es decir, dados dos enteros, m y n, tales que $\mathrm{mcd}(m, n) = 1$, entonces $\phi(m \cdot n) = \phi(m) \cdot \phi(n)$.
3. Si $n = p^e$, entonces $\phi(n) = \phi(p^e) = (p-1)p^{e-1}$.
4. Si $n = p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}$, de las anteriores propiedades se deduce que
$$\phi(n) = n \prod_{i=1}^{k} \left(1 - \frac{1}{p_i}\right)$$
5. Si n es un entero, entonces
$$\sum_{d|n, d>0} \phi(d) = n$$

A continuación se muestran algunos ejemplos de cálculo del indicador de Euler contando manualmente todos los enteros del anillo que son relativamente primos.

Ejemplo 3.1 *Sea $m = 6$. El conjunto asociado sería, $\mathbb{Z}_6 = \{0, 1, 2, 3, 4, 5\}$.*

$$\begin{array}{lcllcl} \text{mcd}(0,6) & = & 6 & \text{mcd}(3,6) & = & 3 \\ \text{mcd}(1,6) & = & 1 & \text{mcd}(4,6) & = & 2 \\ \text{mcd}(2,6) & = & 2 & \text{mcd}(5,6) & = & 1 \end{array}$$

Como existen dos números en el conjunto que son primos relativos con 6 (1 y 5), la función $\phi(6) = 2$. ■

Ejemplo 3.2 *Sea $m = 5$. El conjunto asociado sería $\mathbb{Z}_5 = \{0, 1, 2, 3, 4\}$.*

$$\begin{aligned} \text{mcd}(0,5) &= 5 \\ \text{mcd}(1,5) &= 1 \\ \text{mcd}(2,5) &= 1 \\ \text{mcd}(3,5) &= 1 \\ \text{mcd}(4,5) &= 1 \end{aligned}$$

Encontramos 4 números que son relativamente primos a 5, por lo que $\phi(5) = 4$, el resultado sería directo si aplicamos la propiedad (1), Si p es un primo, entonces $\phi(p) = p - 1$. ■

De estos ejemplos podemos concluir que el cálculo del indicador de Euler ϕ para los números grandes que se requieren en la criptografía de clave pública sería extremadamente si no se aplican las propiedades estudiadas. Veamos el siguiente ejemplo:

Ejemplo 3.3 *Dado un número primo $p = 131071$, se tiene $\phi(131071) = 131070$. Por su parte, como $n = 792$ es compuesto y $729 = 2^3 \cdot 3^2 \cdot 11$, resulta que*

$$\phi(792) = 792 \cdot \left(1 - \frac{1}{2}\right)\left(1 - \frac{1}{3}\right)\left(1 - \frac{1}{11}\right) = 240 \quad \blacksquare$$

Es importante destacar que para calcular de esta forma el indicador de Euler ϕ, necesitamos conocer la factorización de n. Esta propiedad es la base del esquema de clave pública RSA: si se conoce la factorización de un número determinado, podemos calcular el indicador de Euler ϕ y descifrar el texto cifrado. Si no sabemos la factorización, no puede calcularse la función de Euler y, por lo tanto, no se puede descifrar el texto cifrado.

3.1.6. TEOREMA DE EULER Y PEQUEÑO TEOREMA DE FERMAT

Alrededor de 1636, Pierre de Fermat enunció este teorema que aparece en una de sus cartas a su confidente Frénicle de Bessy, fechada el 18 de octubre de 1640, con el siguiente texto: p divide a $a^{p-1} - 1$ cuando p sea primo y a sea coprimo con p. Como de costumbre, omitió la demostración del mismo. Como nota curiosa cito el párrafo de la carta:

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

> Todo número primo mide una de las potencias menos uno de cualquier progresión en la que el exponente es un múltiplo del primo dado menos uno. (...) Y esta proposición es generalmente cierta para todas las progresiones y todos los números primos; le enviaría la prueba, si no temiese que es demasiado larga...

El pequeño teorema de Fermat es uno de los teoremas clásicos de teoría de números relacionado con la divisibilidad. Es útil para las pruebas de primalidad y en muchos otros aspectos de la criptografía de clave pública. Se formula de la siguiente manera:

Teorema 3.4 (Pequeño Teorema de Fermat) *Si p es un número primo, para todo $a \in \mathbb{Z}_p$ se verifica:*

$$a^p \equiv a (\text{mód } p)$$

y si a no es divisible por p, entonces:

$$a^{p-1} \equiv 1 (\text{mód } p)$$

Observamos que la aritmética en cuerpos finitos $GF(p)$ se realiza modulo p, y por lo tanto, el teorema se cumple para todos los enteros que son elementos de un cuerpo finito $GF(p)$. El teorema se utiliza ampliamente en criptografía.

Una aplicación directa del mismo es el cálculo de la inversa en un cuerpo finito. Podemos reescribir la ecuación como, $a \cdot a^{p-2} \equiv 1 (\text{mód } p)$ que es exactamente la definición de inverso multiplicativo. Así, inmediatamente tenemos una manera de invertir un entero modulo un primo:

$$a^{-1} \equiv a^{p-2} \text{ mód } p. \tag{3.1}$$

Este método de inversión se aplica solamente si p es un número primo. La realización de la exponenciación que se muestra en la Ec. (3.1) es generalmente más lenta que si se utiliza el Algoritmo euclidiano extendido. Sin embargo, hay situaciones donde es ventajoso usar el pequeño Teorema de Fermat, por ejemplo, en tarjetas inteligentes u otros dispositivos que tienen un acelerador de hardware para la exponenciación rápida. Esto no es inusual porque muchos algoritmos de clave pública requieren la exponenciación, como veremos en capítulos posteriores. Una generalización del pequeño Teorema de Fermat para cualquier módulo entero, es decir, módulos que no son necesariamente primos, es el Teorema de Euler que trataremos brevemente a continuación.

Teorema 3.5 (Teorema de Euler) *Para todo elemento $a \in \mathbb{Z}_n^*$ se verifica*

$$a^{\phi(n)} \equiv 1 (\text{mód } n)$$

3.2. CRIPTOSISTEMA RSA

RSA es el más usado de los algoritmos asimétricos. Debe su nombre a sus tres inventores: Ronald Rivest, Adi Shamir y Leonard Adleman, que publicaron por primera vez el método RSA en 1977. Sus dos claves sirven indistintamente tanto para cifrar como para autentificar.

El RSA, que basa su seguridad en la dificultad computacional de factorizar un número compuesto muy grande, producto de dos primos grandes.

Dado un número entero positivo n, calcular su factorización como producto de números primos:

$$n = p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}$$

donde cada p_i es primo, $p_1 < \ldots < p_k$ y cada e_i es un entero positivo.

Este problema tiene una complejidad algorítmica similar a la del cálculo de logaritmos discretos y ambos son considerados intratables con la capacidad de cómputo actual cuando se utilizan números por encima de los miles de bits. El mejor algoritmo conocido para resolver este problema es la *criba general de cuerpos numérico*, cuyo tiempo de ejecución es subexponencial.

Las claves pública y privada se calculan a partir de un número que se obtiene como producto de dos primos grandes. Un atacante que quiera recuperar un texto claro a partir del criptograma y de la clave pública, tiene que enfrentarse al problema de la intratabilidad de la factorización de enteros grandes.

El RSA estuvo patentado por los Laboratorios RSA hasta el 20 de septiembre de 2000, de modo que su uso comercial estuvo restringido hasta esa fecha. Actualmente el RSA se implementa en muchos sistemas comerciales. Es utilizado por los servidores web y navegadores para asegurar el tráfico web, se utiliza para garantizar la privacidad y autenticidad del correo electrónico, se usa para asegurar el inicio de sesión remota y es el corazón de los sistemas de pago con tarjeta de crédito electrónica. En resumen, RSA se utiliza con frecuencia en aplicaciones donde la seguridad de los datos digitales es crucial. Actualmente los modos de utilización del RSA está recogido en el estándar PKCS #1 V2.2: RSA CRYPTOGRAPHY STANDARD. Como todo criptosistema de clave pública, tiene tres partes: generación de claves, cifrado de mensajes y descifrado de mensajes.

3.2.1. GENERACIÓN DE CLAVES DEL RSA

Antes de comenzar el proceso de comunicación, cada participante, deberá crear su clave pública y su correspondiente clave privada, para ello deben seguir el Protocolo 3.2. Siendo e el *exponente de cifrado* y d *exponente de descifrado*, y n es el *módulo del criptosistema RSA*. Un mensaje en claro es un número entero comprendido entre 1 y n, lo que significa que es un entero módulo n.

Los valores de p, q y $\phi(n)$ deben permanecer secretos, aunque no se consideren parte de la clave privada, ya que no son necesarios para el descifrado. Típicamente se eligen exponentes de cifrado e, pequeños ($e = 3 = 2 + 1$, ó $e = 2^{16} + 1 = 65537$) con el objetivo de que su expresión binaria sólo requiera de unas pocas operaciones matemáticas haciendo más eficiente el criptosistema. La operación computacional más costosa en este criptosistema es la exponenciación.

3.2.2. CIFRADO DE MENSAJES RSA

Un mensaje es un entero $M \in \mathbb{Z}_n^*$. Para cifrar M generalmente se calcula $C = M^e \bmod n$. Para descifrar el texto cifrado el receptor legítimo calcula $C^d = M^{ed} = M \bmod n$. En la práctica, los mensajes se rellenan antes del cifrado utilizando alguna aleatoriedad. Por ejemplo, un algoritmo de relleno simple (pero insuficiente) puede rellenar un texto claro M agregando algunos bits aleatorios a uno de los extremos antes del cifrado. Agregar aleatoriedad al proceso de cifrado es necesario

Protocolo 3.2 Protocolo de generación de claves RSA

1. Generar aleatoriamente dos números primos grandes y calcular $n = p \cdot q$, (del mismo tamaño $n/2$ cada uno). Calcular el orden del grupo multiplicativo $\mathbb{Z}_n^*$, usando el *indicador de Euler*,
$$\phi(n) = (p-1)(q-1).$$
2. Elegir un entero positivo e, $2 < e < \phi(n)$, que sea primo con $\phi(n)$:
$$\text{mcd}(e, \phi(n)) = 1.$$
3. Usando el algoritmo de Euclides extendido, calcular el inverso, d, de e en $\mathbb{Z}_{\phi(n)}$: el único entero $1 < d < \phi(n)$, tal que:
$$e \cdot d \equiv 1 \text{ mód } \phi(n).$$
4. La clave pública de A es el par (n, e) y su clave privada es d.

para una garantizar la seguridad. Si Bob, quiere enviar un mensaje M cifrado a Alice, seguirá el Protocolo 3.3.

Protocolo 3.3 Protocolo cifrado RSA

1. Obtener la clave pública de Alice: (n_A, e_A).
2. Codificar M como un elemento, m, de $\mathbb{Z}_{n_A}^* = \{0, 1, \ldots, n_A - 1\}$.
3. Calcular el valor del mensaje cifrado y enviarlo a Alice: $c = m^{e_A} \text{ mód } n_A$.

Recuerde que, $m \in \mathbb{Z}_{n_A}^*$ ya que si m no es primo con n, entonces m es divisible por p ó por q. Por ejemplo, si $m = p \cdot k$ para algún entero k, entonces el cálculo de c permitiría factorizar n, calculando $c = m^e \text{ mód } n$, de donde resultaría que $\text{mcd}(c, n) = p$, de este modo se rompe el criptosistema, pues una vez que se conoce p, se calculan $q = n/p$, y $\phi(n) = (p-1)(q-1)$.

A partir de estos valores y del exponente de cifrado, e, se calcula la clave privada d aplicando el algoritmo de Euclides extendido. Si el mensaje m es un entero mayor que n, entonces m debe descomponerse en varios mensajes de menor longitud, siendo cada uno de ellos sea un entero de $\mathbb{Z}_n^*$.

3.2.3. DESCIFRADO DE MENSAJES RSA

Cuando Alice, recibe el criptograma c, recupera el mensaje original m, mediante el Protocolo 3.4.

Protocolo 3.4 Protocolo de descifrado RSA

Utilizando su clave privada d_A Alice calcula:

$$\begin{aligned} c^{d_A} &\equiv (m^{e_A})^{d_A} \text{ mód } n_A \equiv m^{e_A d_A} \text{ mód } n_A \\ &\equiv m \text{ mód } n_A. \end{aligned}$$

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

CONMUTATIVIDAD DEL CIFRADO Y DESCIFRADO EN RSA

Por las propiedades de la exponenciación modular, el cifrado y descifrado son conmutativos: $M = (M^e \bmod n)^d \bmod n = M^{d.e} \bmod n = (M^d \bmod n)^e \bmod n = M$.

Observe que si ciframos M con la clave pública e y a continuación desciframos el resultado con la privada d obtenemos de nuevo M, además sucede que si ciframos M con la clave privada d y desciframos con la clave pública e, volveremos a obtener M.

Esta propiedad es importante porque nos permite utilizar RSA no sólo para cifrar un mensaje, sino también para autenticar el mensaje, como se verá más adelante. Los algoritmos que se emplean para generar dos primos grandes, pueden consultarlos en [8, 19, 20].

El término "número primo grande", debe entenderse en el sentido de que los métodos de factorización existentes en cada momento, no permitan factorizar el producto $p \cdot q$. En la actualidad, se recomienda que los primos p y q tengan como mínimo 512 bits cada uno.

3.2.4. GENERACIÓN DE CLAVES Y DESCIFRADO CON RSA-CRT

La implementación del RSA en tarjetas inteligentes conocida como RSA-CRT, permite realizar los cálculos de forma más rápida utilizando el Teorema Chino del resto 2.7 (CRT, *Chinese Remainder Theorem*). Los protocolos de generación de claves (ver Protocolo 3.5) y de descifrado son ligeramente diferentes al presentado anteriormente (ver Protocolo 3.6). El protocolo de descifrado con RSA-CRT es mucho más rápido que la exponenciación modular [21].

A continuación mostramos el protocolo de generación de claves que emplea el Teorema Chino de los restos para agilizar la carga computacional de algoritmo.

Protocolo 3.5 Protocolo de generación de claves RSA-CRT

1. Generar dos primos aleatorios grandes p y q.
2. Calcular $n = p \cdot q$, y $\phi(n) = (p-1)(q-1)$.
3. Seleccionar dos enteros aleatorios d_p, d_q tales que,
$$\mathrm{mcd}(d_p, p-1) = 1, \quad \mathrm{mcd}(d_q, q-1) = 1, \quad d_p \equiv d_q \bmod 2.$$
4. Aplicando el CRT, calcular un entero d tal que,
$$d \equiv d_p \bmod p-1 \equiv d_q \bmod q-1.$$
5. Calcular $e \equiv d^{-1} \bmod \phi(n)$.

PROTOCOLO DE DESCIFRADO RSA-CRT

Este protocolo utiliza también el Teorema Chino del resto para agilizar los cálculos, a continuación se describe el protocolo.

Protocolo 3.6 Protocolo de descifrado RSA-CRT

Para recuperar el texto claro, m, Alice usa su clave privada, d, y calcula

1. $m_p = c^{d_p} \bmod p = c^d \bmod p$; y $m_q = c^{d_q} \bmod q = c^d \bmod q$.
2. Mediante el CRT, determina una solución a las ecuaciones:
   $m \equiv m_p \bmod p$
   $m \equiv m_q \bmod q$

3.2.5. SEGURIDAD DEL RSA

Desde su publicación inicial, muchos investigadores han analizado las vulnerabilidades del criptosistema RSA. Aunque en más de veinte años de investigación se han encontrado una serie de ataques fascinantes, ninguno de ellos es devastador. En su mayoría ilustran los peligros del uso incorrecto del RSA. Como ejemplo más vivo tenemos el reciente ataque a la implementación del módulo de generación de claves de RSA denominado ataque ROCA [22] que retomaremos más adelante. Como puede verse, la implementación segura del RSA no es una tarea trivial.

Existen dos importantes resultados relacionados con la seguridad del RSA. 1)En [23] se demostró que calcular la clave secreta del RSA es equivalente a factorizar el módulo por medio de un algoritmo de tiempo polinómico probabilístico. 2) Más tarde en [24] se demuestra que existe otro algoritmo equivalente a factorizar el módulo, pero de tiempo polinómico.

El objetivo fundamental de un criptoanalista al romper un criptosistema es obtener la clave privada de un usuario. Como sabemos, la seguridad del RSA está basada en la dificultad computacional del problema de factorizar números enteros.

Los ataques a la seguridad se dividen en las siguientes categorías:

1. Ataques relativos a la factorización del módulo: Consiste en tratar de factorizar el módulo n del criptosistema. Sabemos que todo número entero puede factorizarse como producto de números primos y que si el número es grande es computacionalmente difícil obtener su factorización (*Teorema Fundamental de la Aritmética*). Por tanto, este problema se considera muy difícil computacionalmente; aunque existen ciertas clases especiales de números enteros que sí pueden factorizarse eficientemente por medio de algoritmos específicos [19, 25].

   Una vez que el atacante factoriza el módulo n y obtiene sus factores primos p y q, la seguridad del sistema queda comprometida. Efectivamente, una vez obtenidos los primos p y q el atacante ejecutaría algoritmo de generación de claves: calcular el indicador de Euler $\phi(n) = (p-1)(q-1)$, paso 1 del Protocolo 3.2 conoce el exponente de cifrado e que es público, y calcula su inverso d módulo $\phi(n)$, paso 3 del Protocolo 3.2 empleando el *Algoritmo de Euclides Extendido*, para obtener la clave privada d.

   - **Recomendación 1**: Los primos p y q deben ser números primos grandes y de la misma longitud (el mismo número de bits) aproximadamente. Actualmente se recomienda que p y q tengan cada uno una longitud mínima de 512 bits, por tanto, n tendrá al menos, 1024 bits (alrededor de 309 dígitos).
   - **Recomendación 2**: La diferencia entre p y q debe ser un número grande (que su diferencia sea un número grande), ya que se podría factorizar n y la seguridad del sistema estaría

comprometida. Estas dos recomendaciones (1 y 2) dificultan en extremo los ataques basados en los algoritmos detallados en [19, 25].

- **Recomendación 3**: El máximo común divisor de $p-1$ y $q-1$ debe ser pequeño. Esta condición se consigue si p y q son primos 1-seguros. Un primo p es 1-seguro si el entero $(p-1)/2$ también es primo.
- **Recomendación 4**: Los primos p y q deben ser robustos y cumplir que $p-1, p+1$ contienen un factor primo grande r. Existen varios algoritmos para generar primos robustos, ejemplo de estos se detallan en [26].

2. Ataques a exponentes de cifrado pequeño: Sabemos que para hacer más eficiente el cifrado del RSA los exponentes de cifrado e se eligen pequeños. En el caso que se envíen e mensajes cifrados del mismo mensaje en claro m con el mismo exponente de cifrado pequeño e, a usuarios cuyos módulos no son primos entre sí dos a dos, $\mathrm{mcd}(n_i, n_j) = 1$ para $i < j$, el criptoanalista podría recuperar el mensaje m por medio del algoritmo de Gauss [27, 8, 28]. La seguridad global del sistema, no se ve comprometida ya que el ataque no permite hallar el exponente de descifrado d, ni factorizar su módulo n. Esta vulnerabilidad aparece cuando el exponente de cifrado es $e = 3$ si se conocen dos tercios del mensaje ([29]).

3. Ataques a exponentes de descifrado pequeños: Ataques reportados en Wiener-Boneh-Durfee [30, 31, 32] entre otros. Para evitarlos habría que seleccionar d de modo que su tamaño sea lo más próximo posible al módulo n. Pero dada la estrecha relación entre los valores de d y e, el valor de uno condiciona el valor del otro. En [33] se propone un algoritmo para la elección de las claves que afecta la elección de los factores primos del módulo n, pero no a su tamaño, por lo que no pone en peligro el criptosistema contra los ataques por factorización.

4. Ataques cíclicos y sus generalizaciones: Las operaciones en el RSA se realizan dentro de un cuerpo finito, por tanto si multiplicamos dos números repetidamente, obtendremos finalmente el resultado inicial. Los ataques cíclicos permiten recuperar el texto en claro, pero no rompen el criptosistema. Este ataque es muy lento e ineficiente si se cumple la recomendación 4 arriba mencionada. Los ataques cíclicos generalizados sí pueden comprometer la seguridad del criptosistema RSA, aunque, dada la dificultad de la factorización de números enteros, no suponen una amenaza real, siempre y cuando los factores p y q se elijan cumpliendo las recomendaciones explicadas.

5. Mensajes inocultables: Este ataque tiene éxito siempre y cuando la potencia e-ésima del texto en claro coincida con el propio texto en claro, es decir aquellos que verifican $m^e \equiv m \mod n$. Este ataque se aplica a ciertos valores del módulo n; si p y q se toman de modo aleatorio y, e y $\phi(n)$ son primos entre sí (es decir, $\mathrm{mcd}(e, \phi(n)) = 1$), la probabilidad de encontrar un mensaje inocultable es menor que $K(\ln n)^2/n$, siendo K un valor constante.

   A pesar de estos ataques se puede decir que el sistema de cifra RSA permite no sólo garantizar la confidencialidad de la comunicación entre dos partes, cifrando en origen el mensaje que se va a transmitir por un canal inseguro y descifrándolo en recepción, sino que también proporciona otros servicios o funciones de seguridad de la información, como son la autenticación de origen, la integridad o el no-repudio mediante la firma digital, temas que será tratado en otra lección.

3.3. RESUMEN

Hemos estudiado que, el algoritmo de Euclides permite hallar el mcd de dos números enteros realizando una reducción de los operandos de forma recursiva, además que el algoritmo Extendido de Euclides permite el cálculo en modulo inverso de enteros, lo que es de gran importancia en la criptografía de clave pública, particularmente en los sistemas RSA. La función ϕ de Euler es muy importante, dado un número entero positivo n nos da el número de enteros positivos menores o iguales a n y coprimos con n, lo que es fundamental en la definición del sistema de cifrado RSA. El pequeño teorema de Fermat está relacionado con la divisibilidad, nos sirve para las pruebas de primalidad y en muchos otros aspectos de la criptografía de clave pública, siendo el Teorema de Euler una generalización del Pequeño Teorema de Fermat. El Algoritmo de Euclides extendido (AEE) se encuentra prácticamente en todas las implementaciones de los sistemas de clave pública, especialmente para realizar la inversión modular. Con respecto a las matemáticas en este capítulo, recomendamos revisar [34], que se encuentra entre los materiales complementarios.

La efectividad de los algoritmos asimétricos depende de una clase de problemas matemáticos conocidos como funciones unidireccionales, que requieren relativamente poca potencia de cálculo para ejecutarse en una dirección, pero muchísima potencia para calcular la inversa. En el caso del RSA el problema matemático en cuestión es el problema de la factorización de enteros. Otro criptosistema basado en este problema es el "criptosistema de Rabin", el alumno interesado en el mismo puede estudiarlo en [35]. Existen un sinfín de criptosistemas de clave pública que no hemos tratado aquí, entre los que destacan los "criptosistemas basados en el problema de la mochila", dos de los criptosistemas de interés serían el Criptosistema de Merkle-Hellman [36] y el Criptosistema de Chor-Rivest [37].

En general los algoritmos de clave pública son computacionalmente costosos (una buena manera de decir que son lentos), y por lo tanto son poco adecuados para el cifrado de grandes cantidades de datos, como por ejemplo, el cifrado de imágenes. Los sistemas de clave pública combinados con los sistemas de clave secreta, proporcionan todas las funciones requeridas por los protocolos de seguridad modernos, se resuelve el intercambio de claves con la clave pública, el no repudio con las firmas digitales y los certificados autentican irrevocablemente a un usuario o entidad con sus claves pública y privada, una vez acordada una clave entre dos entidades, el cifrado de la información que viaja por canales inseguros, se hace con los algoritmos de clave simétrica que son más rápidos que los de clave pública.

Conocer mejor el RSA-1

Ejecuta el siguiente ejemplo del RSA en esta plataforma:
https://www.cs.drexel.edu/~jpopyack/IntroCS/HW/RSAWorksheet.html

1. Elige los números $p = 11$ y $q = 3$, y elige **e**, dentro de todos los posibles valores que te ofrece la plataforma, de modo obtengas como resultado resultado $e = 3$, y $d = 7$,
2. **Observa la comprobación** que hace la plataforma para garantizar que estos valores pueden ser elegidos como par de claves del RSA.
3. Ahora cifra y descifra varios mensajes. Como ya sabemos el RSA cifra números menores que su módulo n.
4. ¿Qué ocurre con el cifrado/descifrado de los mensajes $m = 11$ y $m = 12$?
5. Publica en el foro lo que has encontrado? ¿Cómo se denominan estos mensajes?

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

Ejecuta el siguiente ejemplo del RSA en esta plataforma:
https://www.cs.drexel.edu/~jpopyack/IntroCS/HW/RSAWorksheet.html

1. Elige los números primos $p = 383$ y $q = 521$
2. De los K valores posibles que le ofrece la plataforma, elija $K = 397281$ y calcule sus factores primos.
3. Elija $e =$ al factor primo más pequeño y calcule d. Compruebe que verifican los criterios que deben cumplir.
4. Descifra este número: 175616. Publica en el foro tu resultado.
5. Cifra tu edad y envía al foro: el texto cifrado, el valor de e, d elegido.
6. Cuál de los parámetros anteriores no debes revelar para cuidar tu privacidad?
7. Publica en el foro lo que has aprendido.

En los materiales complementarios se tienen algunos resúmenes de cada uno de los temas explicados, que esperamos sean de gran utilidad.

4. CRIPTOGRAFÍA CON CURVAS ELÍPTICAS

En la Sec. 2 se definió el criptosistema de ElGamal para el grupo $\mathbb{Z}_p^*$, siendo p un número primo cuya seguridad se fundamenta en la dificultad de resolver el problema del logaritmo discreto (DLP). La definición de este criptosistema puede extenderse a grupos cíclicos más generales. De hecho se han propuesto algunos grupos alternativos para la definición de nuevos criptosistemas, cuya seguridad se basa en un problema de características similares al DLP. Uno de los grupos que ha recibido gran atención en los últimos años, es el grupo de puntos de una curva elíptica (EC) definida sobre un cuerpo finito.

El uso de las curvas elípticas (EC) sobre cuerpos finitos se debe fundamentalmente a Neal Koblitz y Victor Miller quienes las propusieron de forma independiente en 1987 para implementar algunos criptosistemas conocidos, actualmente denominada como Criptografía con Curva Elíptica (*Elliptic Curve Cryptography*).

La criptografía con curvas elípticas (ECC) se caracteriza por emplear claves más cortas que las utilizadas en otros criptosistemas como el RSA y ElGamal, para un mismo nivel de seguridad, véase el cuadro 1.1. Esto la convierte en un criptosistema ideal para sistemas con pocos recursos hardware/software, es decir, la utilización de menos memoria y elementos de hardware de menor tamaño, como son los teléfonos móviles, las tarjetas inteligentes, los sistemas de identificación por radio frecuencia RFID, los sistemas de votación electrónica, etc.

Para conocer el funcionamiento y la seguridad que brinda la ECC, es necesario comprender qué son las curvas elípticas, y cómo se utilizan en criptografía. Algunos aspectos matemáticos necesarios para entender la criptografía con curvas elípticas, son los conceptos de estructuras de grupo y cuerpo y la aritmética modular mostrados en la Sec. 2.1, la descripción matemática completa de la teoría de las curvas elípticas puede estudiarse en [7]. Estudiaremos el problema del logaritmo discreto con curvas elípticas y su aplicación en la criptografía, así como las principales ventajas que tienen estos criptosistemas basados en EC. También analizaremos someramente la generación de claves y algoritmos de cifrado y descifrado de estos criptosistemas. Si se desea profundizar en el tema recomendamos revisar los contenidos utilizando la bibliografía citada en esta sección. Todos los gráficos sobre curvas elípticas fueron generados usando el software CoCalc(SageMathCloud).

4.1. ALGUNOS ASPECTOS MATEMÁTICOS

Los criptosistemas de curvas elípticas, definen la curva elíptica sobre un cuerpo finito de Galois $GF(q)$ siendo q un número primo o la potencia de un número primo. Aunque su estructura algebraica es algo compleja, su implementación suele resultar en ocasiones más eficiente que la aritmética modular.

Una curva elíptica definida sobre un cuerpo arbitrario $\mathbb{K}$ es una curva plana *sin puntos singulares* que viene dada por la ecuación general de Weierstrass,

$$y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6, \quad a_i \in \mathbb{K}. \tag{4.1}$$

junto con un elemento simple denotado por O, denominado punto en el infinito.

Si el cuerpo $\mathbb{K}$ es el de los número reales, entonces la curva elíptica definida sobre los números reales es una curva cúbica definida en el plano por una ecuación de la forma $y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6$, y sus infinitos puntos (x, y) tendrán como coordenadas números reales, que

deben cumplir la condición de que la curva no presente picos ni se cruce sobre sí misma (que la curva no tenga singularidades).

Aunque no es posible dibujar imágenes significativas de curvas elípticas, intuitivamente es útil pensar en términos de gráficos sobre los números reales. En la Fig. 5 se muestran las gráficas de dos curvas elípticas sobre $\mathbb{R}$. La primera tiene tres puntos en el eje de las abscisas mientras que la segunda tiene uno. Observe además, que la curva es simétrica respecto del eje de abscisas.

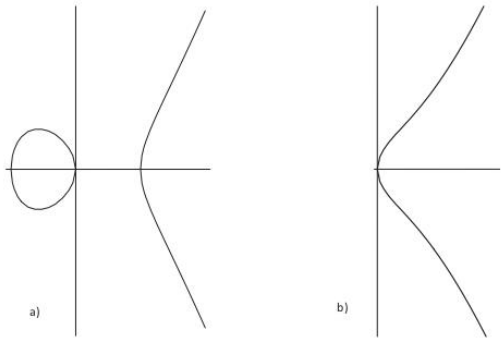


Figura 5: Curvas elípticas: a) $y^2 = x^3 - x$, b) $y^2 = x^3 + x$, definidas sobre $\mathbb{R}$.

CURVAS ELÍPTICAS COMO GRUPO ABELIANO

Una de las características que permite utilizar curvas elípticas en Criptografía es que sus puntos pueden definir una estructura de grupo mediante la operación de suma. Para definir un grupo, necesitamos definir una operación, en este caso la suma denotada por $+$, para el conjunto $E(a, b)$, donde a y b satisfacen la condición de que la curva no tenga singularidades. En términos geométricos las reglas para la suma sería: si tres puntos en una curva elíptica caen sobre una línea recta, su suma es O. A partir de esta definición podemos definir las reglas para la suma sobre una curva elíptica:

1. $O + O = O$ elemento neutro de la suma. Para cualquier punto P sobre la curva elíptica $P + O = P$.
2. Si $P = (x, y)$, entonces $-P = (x, -y)$.
3. Para sumar dos puntos P y Q con diferentes coordenadas x usamos *método de la cuerda y la tangente*, se dibuja una línea recta entre los puntos, el tercer punto de intersección R, es un punto único. Para formar una estructura de grupo, necesitamos definir la suma sobre estos tres puntos: $P + Q = -R$. Es decir, definimos $P + Q$ como la imagen especular (con respecto al eje x) del tercer punto de intersección. La Fig. 6 muestra esta construcción.
4. La interpretación geométrica anterior se aplica también a dos puntos, P y $-P$, con la misma coordenada x. Los puntos se unen por una línea vertical, que se puede ver como la intersección de la curva con el punto infinito. Por tanto, $P + (-P) = O$, que es consistente con el punto (2).
5. Para doblar un punto P, dibujamos una línea tangente y encontramos el otro punto de intersección R. Entonces el resultado sería $P + P = 2P = -R$.

Las reglas anteriores demuestran que el conjunto $E(a,b)$ es un grupo abeliano. Si consideramos el conjunto de puntos $E(a,b)$ junto con el elemento O, que satisfacen la Ec. (4.1) sin que existan singularidades, y utilizando valores diferentes que satisfagan esta ecuación obtendremos diferentes curvas. La Fig. 6 muestra la curva $E(1,1)$ donde en la figura de la izquierda hemos sumado dos puntos $P+Q$, y en la de la derecha hemos doblado el punto P.

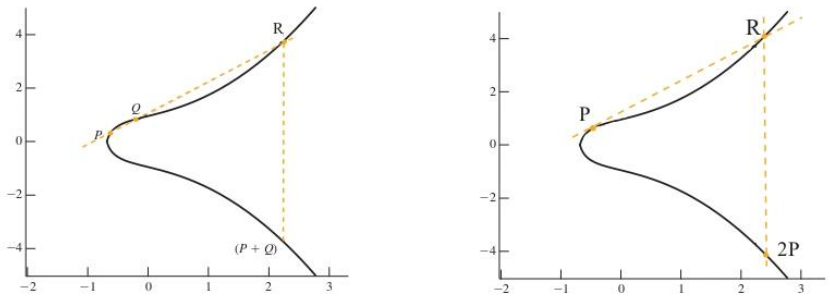


Figura 6: Suma de puntos en la curva elíptica $y^2 = x^3 + x + 1$ sobre $\mathbb{R}$.

Si se considera como cuerpo base un cuerpo finito, los coeficientes de la ecuación y las coordenadas de los puntos de la curva pertenecerán a dicho cuerpo y la ecuación de Weierstrass dada en (4.1) se puede simplificar en función de la característica de un determinado cuerpo finito ([38]).

En el caso de una curva elíptica sobre un cuerpo finito, $\mathbb{F}_q$, con $q = p^r$, $r \geq 1$ representada por $E(\mathbb{F}_q)$, si la característica del cuerpo es distinta de 2 y de 3, $(p \neq 2, 3)$, a partir de la Ec. (4.1), operando matemáticamente se puede obtener la ecuación reducida de Weierstrass como:

$$y^2 = x^3 + ax + b, \quad a, b \in \mathbb{F}_p. \tag{4.2}$$

El discriminante del polinomio cúbico en x será distinto de cero, es decir,

$$\Delta = 4a^3 + 27b^2 \neq 0, \tag{4.3}$$

para que la curva no tenga singularidades[a].

[a]Esto significa que siempre se puede calcular la tangente en cada punto de la curva.

En la Figura 7 se muestra conjunto de puntos afines de la curva elíptica definida por la ecuación $y^2 = x^3 + x + 1$ en este caso, utilizando el cuerpo finito de $p = 23$ elementos, $\mathbb{F}_{23}$. Puede verse, su representación gráfica no parece una "curva", a pesar de serlo; esto es así porque estamos trabajando sobre un cuerpo finito. Observe cómo los puntos cuya coordenada x es diferente de cero se distribuyen simétricamente respecto de una línea horizontal que pasa por el punto $\frac{p}{2} = \frac{23}{2}$.

Ejemplo 4.1 *Ejemplo sobre* $\mathbb{Z}_{23}$.
Dado $p = 23$ *y la curva* $y^2 = x^3 + x + 1$, *(* $a = 1; b = 1$*).*

$$E(\mathbb{Z}_{23}) = \{f(x,y) : y^2 = x^3 + x + 1\} \cup \{O\}.$$

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

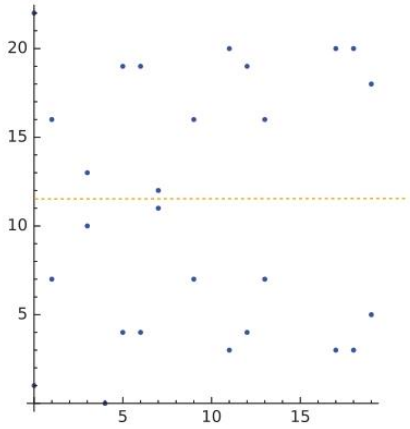


Figura 7: Curva elíptica $y^2 = x^3 + x + 1$, definida sobre $\mathbb{F}_{23}$.

La solución sería:

$(0,1)$	$(5,4)$	$(9,16)$	$(17,3)$
$(0,22)$	$(5,19)$	$(11,3)$	$(17,20)$
$(1,7)$	$(6,4)$	$(11,20)$	$(18,3)$
$(1,16)$	$(6,19)$	$(12,4)$	$(18,20)$
$(3,10)$	$(7,11)$	$(12,19$	$(19,5)$
$(3,13)$	$(7,12)$	$(13,7)$	$(19,18)$
$(4,0)$	$(9,7)$	$(13,16)$	

Como se puede ver, todos ellos verifican la ecuación que define a la curva considerando la operación definida en el conjunto $\mathbb{Z}_{23}$, que consiste en multiplicar los números y luego tomar como resultado el resto de la división entera del producto hallado entre 23.■

Cuando el cuerpo base es un cuerpo finito, la suma de dos puntos se realiza usando las coordenadas de los puntos dados. Así, si $P = (x_1, y_1)$ y $Q = (x_2, y_2)$, el resultado de su suma $P + Q = R$ es el punto $R = (x_3, y_3)$, y se calcula como sigue:

$$\begin{aligned} x_3 &= \lambda^2 - x_1 - x_2, \quad \text{si } x_1 \neq x_2 \\ y_3 &= \lambda(x_1 - x_3) - y_1, \quad \text{si } x_1 = x_2;\ y_1 \neq 0 \end{aligned}$$

donde λ es la pendiente de la recta que une los puntos P y Q si son distintos, o la pendiente de la recta tangente al punto si éstos coinciden.

$$\lambda = \begin{cases} \dfrac{3x_1^2 + a}{2y_1}, & \text{si } P = Q \\ \dfrac{y_2 - y_1}{x_2 - x_1}, & \text{si } P \neq Q \end{cases}$$

siendo a el coeficiente de la curva.

Dado que el neutro de la suma es el punto del infinito, O, se verifica que si $P = (x_1, y_1)$, su opuesto es $-P = -(x_1, y_1) = (x_1, -y_1)$ (pueden interpretarse como simétricos) y, además, se tiene que $P + (-P) = O$.

Observación 4.1 *Los puntos que pertenecen a la curva son los pares de elementos* (x, y) *que, perteneciendo al cuerpo finito, verifican la ecuación de Weierstrass Ec.* (4.2).

Recomendamos el estudio de las propiedades sobre el cardinal y la estructura de estos grupos, que permiten que sean utilizados en muchas aplicaciones criptográficas.

CURVAS ELÍPTICAS SEGURAS EN CRIPTOGRAFÍA

Si queremos definir un criptosistema basado en una curva elíptica, para determinar qué curva elíptica utilizar, es saludable recurrir a las recomendaciones propuestas en los estándares como por ejemplo [39], y en FIPS186-4. Si se quiere generar una nueva curva elíptica aún siguiendo las recomendaciones de los estándares, existe la dificultad de determinar su número de puntos.

Uno de los teoremas más importantes desde el punto de vista de la criptografía que nos ayuda a resolver el problema es el siguiente,

Teorema 4.1 (Teorema de Hasse) *El número de puntos de una curva elíptica,* $\#E$*, definida sobre un cuerpo finito* F_p*, de* p *elementos es* $\#E(\mathbb{F}_p) = p + 1 - t$*, donde* $|t| \leq 2\sqrt{p}$*, entonces* $\#E(\mathbb{F}_p) \approx p$

Una consecuencia importante de este teorema es que podemos escoger los puntos P uniforme y aleatoriamente sobre una curva elíptica $E(F_p)$ en un tiempo polinomial usando el algoritmo de Schoof [40].

La seguridad de una curva elíptica desde el punto de vista criptográfico depende principalmente de la cardinalidad de la curva (el número de puntos racionales en la curva o el orden del grupo de puntos racionales en la curva). Es por ello que se recomienda que la cardinalidad tenga un factor primo grande para prevenir ciertos ataques.

Un grupo de curva elíptica se denomina criptográficamente fuerte si se considera que el ECDLP subyacente no puede resolverse computacionalmente para una aplicación dada. Los grupos de curvas elípticas criptográficamente fuertes para diferentes niveles de seguridad son publicados por varios organismos de estandarización (por ejemplo, ANSI, ISO, IETF, NIST).

4.2. EL PROBLEMA DEL LOGARITMO DISCRETO CON CURVAS ELÍPTICAS

Definición 4.1 (ECDLP (Elliptic Curve Discrete Logarithm Problem)) *Dada una curva elíptica* E *sobre* $\mathbb{F}_p$*, un generador* P *de un subgrupo cíclico* G *y un punto* Q *de* G*, encontrar el entero* d *tal que* $Q = d \cdot P$*. El entero* d *es el logaritmo discreto de* Q *en base* P *y se escribe,* $d = \log_P Q$.

Como en el caso del problema del logaritmo discreto, se puede calcular $Q = d \cdot P$ de manera eficiente conocidos P y d, en cambio, conocidos P y Q, encontrar d es un problema computacionalmente difícil ([41, 42]).

En el cuadro 4.1 se ilustran los algoritmos más conocidos para resolver el DLP y el ECDLP y su complejidad computacional, siendo n el tamaño del grupo, ya sea un subgrupo cíclico de $\mathbb{F}_p^*$ en el caso del DLP, o un subgrupo cíclico del grupo de puntos de una curva elíptica en el caso del ECDLP. Fuente: Cryp4you

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

Cuadro 4.1: Algoritmos que resuelven el DLP y el ECDLP.

Algoritmo	Complejidad	DLP	ECDLP
Búsqueda exhaustiva	$\mathcal{O}(n)$	si	si
Baby-step Giant-step	$\mathcal{O}(\sqrt{(n)})$	si	si
Pollard-ρ	$\mathcal{O}(\sqrt{(n)})$	si	si
Index-Calculus	$\mathcal{O}(e^{\sqrt{\log p \log\log p}})$	si	no
Number Field Sieve	$\mathcal{O}(e^{1{,}923(\log p)^{\frac{1}{3}}(\log\log p)^{\frac{2}{3}}})$	si	no

4.3. INTERCAMBIO DE CLAVE DE DIFFIE-HELLMAN CON CURVAS ELÍPTICAS (ECDH)

El protocolo de acuerdo de clave de Diffie-Hellman con curvas elípticas (ECDH, Elliptic Curve Diffie-Hellman key agreement protocol) [42] se ha incluido en varios estándares, como el ANSI X9.63 ([43]) y está referenciado en SP 800-56Arv2 del NIST, 2015.

Supongamos que Alice (A) y Bob (B) quieren acordar una clave secreta común usando un canal inseguro. Para ello seleccionan los parámetros de dominio:

1. Se eligen públicamente una curva elíptica E sobre un cuerpo finito $\mathbb{F}$.
2. Elegir aleatoriamente un generador $P = (x_P, y_P)$ de E.

Una vez elegidos los parámetros de dominio se sigue el protocolo siguiente,

Protocolo de Intercambio de claves Diffie y Hellman con curvas elípticas

Alice		**Bob**
	Eligen aleatoriamente en secreto:	
$k_{prA} = a \in \{2, \ldots, \#E - 1\}$		$k_{prB} = b \in \{2, \ldots, \#E - 1\}$
Calcula: $k_{pubA} = aP = A = (x_A, y_A)$		Calcula: $k_{prB} = bP = B = (x_B, y_B)$
	Alice envía: A →	
	← Bob envía: B	
Alice calcula: $k_{AB} = aB = abP$		Bob calcula: $k_{AB} = bA = baP$
	El secreto compartido es $k_{AB} = (x_{AB}, x_{AB})$	

Figura 8: Protocolo de Intercambio de claves de Diffie y Hellman.

El punto calculado, abP, podrá ser utilizado como semilla para determinar una clave que compartirán los dos usuarios como clave secreta común para usar cifrar la comunicación usando el AES [3].

Existe otro protocolo de intercambio de clave, muy extendido e incluido en varios estándares denominado intercambio de claves de Menezes-Qu-Vanstone con curva elíptica (ECMQV) [42].

4.4. CRIPTOSISTEMAS DE CURVAS ELÍPTICAS: ECC

La extrapolación directa de los criptosistemas basados en el problema del logaritmo discreto como ElGamal, a curvas elípticas presentan ciertas vulnerabilidades que han sido analizadas en la literatura ([3, 42]). El Esquema de cifrado integrado de curva elíptica (ECIES) fue propuesto por Abdalla, Bellare y Rogaway [44], y es una variante del esquema de cifrado de clave pública ElGamal. Ha sido estandarizado por varios organismos entre ellos el ANSI X9.63. Se denomina *integrado*, ya que es un esquema híbrido que usa un sistema de clave pública para transportar de manera segura una clave de sesión para utilizarla en un cifrado simétrico.

Existen diferentes variaciones de este algoritmo por lo que estudiaremos el sistema genérico. En el esquema de cifrado ECIES, se comparte un *secreto* usando el algoritmo de intercambio de claves de Diffie-Hellman para derivar dos claves simétricas k_1 y k_2. La clave k_1 se utiliza para cifrar el texto claro usando un cifrado de clave simétrica, mientras que la clave k_2 se usa para autenticar el texto cifrado resultante. Intuitivamente, la autenticación protege contra los ataques de texto cifrado elegidos ya que el adversario no puede generar textos cifrados válidos por sí mismo.

El esquema de un criptosistema ECIES se compone de, un algoritmo de intercambio de claves de Diffie-Hellan con curvas elípticas, un algoritmo de cifrado simétrico y un algoritmo de descifrado simétrico junto con una función hash especial.

En este sistema se utilizan las primitivas criptográficas siguientes:

- KDF es una función de derivación clave (Key derivation function) que se construye a partir de una función hash H. Si se requiere una clave de l bits, KDF(S) se define como la concatenación de los valores hash $H(S, i)$, donde S es la semilla de entrada al generador aleatorio que se utiliza para generar la curva elíptica, donde i es un contador que se incrementa para cada evaluación de la función hash hasta que se hayan generado l bits de valores hash.
- ENC (*Encryption*) es la función de cifrado del criptosistema de clave simétrica usado, por ejemplo, AES. DEC es la función de descifrado.
- MAC (*Message Authentication Code*) es un código de autenticación de mensaje, por ejemplo el HMAC.

Los protocolos de cifrado y descifrado pueden consultarse en [3, 42].

4.5. RESUMEN

La Criptografía de Curva elíptica (ECC) es una variante de la criptografía asimétrica o de clave pública basada en las matemáticas de las curvas elípticas.Los criptosistemas ECC pueden ser más rápidos y usar claves más cortas que otros métodos convencionales, como el RSA, al tiempo que proporcionan un nivel de seguridad equivalente. En las últimas décadas, la criptografía con curvas elípticas ha adquirido una creciente importancia, llegando a formar parte de los estándares industriales. Su principal logro se ha conseguido en los criptosistemas basados en el problema del logaritmo discreto, como los de tipo ElGamal.

Entre las ventajas de estos criptosistemas tenemos:

- El ECDLP es más difícil de resolver que el problema del logaritmo discreto en $\mathbb{Z}_p^*$.
- La ley del grupo es muy fácil de implementar. De hecho, la suma de puntos sólo requiere

unas pocas operaciones aritméticas en el cuerpo finito base $\mathbb{F}_p$ considerado.

- El grupo de una curva tiene aproximadamente, p elementos; de manera que si el mayor factor primo de p es n, entonces el mejor algoritmo conocido para resolver el ECDLP requerirá un tiempo exponencial en n.
- Para un mismo nivel de seguridad los criptosistemas de clave pública con curva elíptica utilizan una clave menor que los criptosistemas basados en el los problemas de factorización de enteros y el logaritmo discreto véase el cuadro 1.1.

A modo de resumen presentamos una tabla con la correspondencia entre los parámetros utilizados en los criptosistemas basados en el logaritmo discreto cuando se trabaja en $\mathbb{Z}_p^*$ y cuando se trabaja en $E(\mathbb{F}_p)$.

Grupo	$\mathbb{Z}_p^*$	$E(\mathbb{F}_p)$
Elementos del grupo	Enteros $1, 2, \ldots, p-1$	Puntos (x, y) sobre E junto con O
Operación del grupo	multiplicación modulo p	Suma de puntos.
Notación	Elementos α, β Multiplicación $\alpha \cdot \beta$ Inverso α^{-1} División α/β Exponenciación α^d	Elementos P, Q Suma $P+Q$ Negativo: $-P$ Resta: $P-Q$ Múltiplo: dP
Problema del logaritmo discreto	Dados $\alpha \in \mathbb{Z}_p^*$ y $\alpha = \beta^d \bmod p$ encontrar d	Dados $P \in E(\mathbb{F}_p)$ y $Q = dP$, encontrar d

Cuadro 4.2: Correspondencia entre $\mathbb{Z}_p^*$ y $E(\mathbb{F}_p)$.

Referencias

[1] W. Diffie and M. E. Hellman. New directions in cryptography. *IEEE Trans. Inform. Theory*, 22:644–654, 1976. 4, 22

[2] R. C. Merkle. Secure communications over insecure channels. *ACM*, 21:294–299, 1978. 5

[3] A Martín F. Montoya y J. Muñoz A. Fuster, L. Hernández. *Criptografía, protección de datos y aplicaciones*. RA-MA, 2012. 5, 7, 23, 47, 48

[4] Victor Shoup. A proposal for an ISO standard for public key encryption. Cryptology ePrint Archive, Report 2001/112, 2001. https://eprint.iacr.org/2001/112. 8

[5] Ronald L. Rivest, Adi Shamir, and Leonard M. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *CACM*, 21(2):120–126, 1978. 10

[6] T. El Gamal. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Trans. Inform. Theory*, 31:469–472, 1985. 10, 26

[7] N. Koblitz. Elliptic curve cryptosytems. *Mathematics of Computation*, 48(177):203–209, 1987. 10, 42

[8] A.J. Menezes. *Elliptic Curve Public Key Cryptosystems*. Springer US, 1993. 10, 37, 39

[9] Victor Shoup. *A Computational Introduction to Number Theory and Algebra*. Cambridge University Press, 2 edition, 2008. 12

[10] C. P. Schnorr. Efficient signature generation by smart cards. *Journal of Cryptology*, 4(3):161–174, Jan 1991. 25

[11] Laurie Law, Alfred Menezes, Minghua Qu, Jerry Solinas, and Scott Vanstone. An efficient protocol for authenticated key agreement. *Designs, Codes and Cryptography*, 28(2):119–134, Mar 2003. 25

[12] Alfred J. Menezes, Paul C. van Oorschot, and Scott A. Vanstone. *Handbook of Applied Cryptography*. CRC Press, 1997. 27, 29, 32

[13] Accredited Standards Committee X9. *Working Draft: American National Standard X9.30-199X: Public Key Cryptography Using Irreversible Algorithms for the Financial Services Industry: Part 1: The Digital Signature Algorithm (DSA)*, June 18, 1993. 28

[14] Douglas R. Stinson. *Cryptography: Theory and Practice*. CRC Press, 1995. 29

[15] H. Riesel. *Prime Numbers and Computer Methods for Factorization*, chapter Prime Numbers and Cryptography, pages 226–238. Birkhauser Boston, 1995. 30

[16] Donald E. Knuth. *Seminumerical Algorithms*, volume 2 of *The Art of Computer Programming*. Addison–Wesley, 1998. Third edition, 1998. 31

[17] Manindra Agrawal, Neeraj Kayal, and Nitin Saxena. Primes is in p. *Annals of Mathematics*, (160):781–793, 2004. 31

[18] Bach E. and Shallit J. *Algorithmic number theory. Vol. 1*. MIT Press, 1996. 31

[19] J. M. Pollard. Theorems on factorization and primality testing. *Proc. Cambridge Philosophical Society*, 76:521–528, 1974. 37, 38, 39

[20] Ronald L. Rivest. Finding four million large random primes. pages 625–626. Springer, 1991. Lecture Notes in Computer Science No. 537. 37

[21] J. J. Quisquater and C. Couvreur. Fast decipherment algorithm for rsa public-key cryptosystem. *Electronics Letters*, 18(21):905–907, 1982. 37

[22] Matus Nemec, Marek Sys, Petr Svenda, Dusan Klinec, and Vashek Matyas. The Return of Coppersmith's Attack: Practical Factorization of Widely Used RSA Moduli. In *24th ACM Conference on Computer and Communications Security (CCS'2017)*, pages 1631–1648. ACM, 2017. 38

[23] Alexander May. Computing the rsa secret key is deterministic polynomial time equivalent to factoring. In Matt Franklin, editor, *Advances in Cryptology – CRYPTO 2004*, pages 213–219, Berlin, Heidelberg, 2004. Springer Berlin Heidelberg. 38

[24] Jean-Sebastien Coron and Alexander May. Deterministic polynomial-time equivalence of computing the rsa secret key and factoring. *Journal of Cryptology*, 20(1):39–50, Jan 2007. 38

[25] H. C. Williams. A $p+1$ method of factoring. *Mathematics of Computation*, 39(159):225–234, 1982. 38, 39

[26] John Gordon. Strong primes are easy to find. In Thomas Beth, Norbert Cot, and Ingemar Ingemarsson, editors, *Advances in Cryptology*, pages 216–223, Berlin, Heidelberg, 1985. Springer Berlin Heidelberg. 39

[27] Johan Hastad. On using rsa with low exponent in a public key network. In *Lecture Notes in Computer Sciences; 218 on Advances in cryptology—CRYPTO 85*, pages 403–408, New York, NY, USA, 1986. Springer-Verlag New York, Inc. 39

[28] Don Coppersmith, Matthew Franklin, Jacques Patarin, and Michael Reiter. Low-exponent rsa with related messages. In Ueli Maurer, editor, *Advances in Cryptology — EUROCRYPT96*, pages 1–9, Berlin, Heidelberg, 1996. Springer Berlin Heidelberg. 39

[29] Don Coppersmith. Small solutions to polynomial equations, and low exponent rsa vulnerabilities. *Journal of Cryptology*, 10(4):233–260, Sep 1997. 39

[30] M. J. Wiener. Cryptanalysis of short rsa secret exponents. *IEEE Transactions on Information Theory*, 36(3):553–558, May 1990. 39

[31] Dan Boneh and Glenn Durfee. Cryptanalysis of RSA with private key d less than n^{292}. In *Proceedings of the 17th International Conference on Theory and Application of Cryptographic Techniques*, EUROCRYPT'99, pages 1–11, Berlin, Heidelberg, 1999. Springer-Verlag. 39

[32] Subhamoy Maitra and Santanu Sarkar. A new class of weak encryption exponents in rsa. In Dipanwita Roy Chowdhury, Vincent Rijmen, and Abhijit Das, editors, *Progress in Cryptology - INDOCRYPT 2008*, pages 337–349, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg. 39

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

[33] Luis Hernández Encinas, Jaime Muñoz Masque, and Araceli Queiruga Dios. Procedimiento y dispositivo de encriptación mediante un criptosistema tipo RSA. Patente Española ES 2 217 959 B1, CSIC, 2004. 39

[34] Victor Shoup. *A Computational Introduction to Number Theory and Algebra*. Cambridge University Press, New York, NY, USA, 2 edition, 2009. 40

[35] M. O. Rabin. Digitalized signatures and public-key functions as intractable as factorization. Technical report, Cambridge, MA, USA, 1979. 40

[36] R. Merkle and M. Hellman. Hiding information and signatures in trapdoor knapsacks. *IEEE Transactions on Information Theory*, 24(5):525–530, 1978. 40

[37] B. Chor and R. L. Rivest. A knapsack-type public key cryptosystem based on arithmetic in finite fields. *IEEE Transactions on Information Theory*, 34(5):901–909, 1988. 40

[38] A. Menezes, Ian F. Blake, XuHong Gao, Ronald C. Mullin, Scott A. Vanstone, and Tomik Yaghoobian. *Applications of Finite Fields*. Springer US, 1993. 44

[39] ANSI. *Public Key Cryptography for the Financial Services Industry: The Elliptic Curve Digital Signature Algorithm (ECDSA)*. American National Standards Institute, National Bureau of Standards, ANSI X9.62, 2005. 46

[40] R. Schoof. Elliptic curves over finite fields and the computation of square roots modulo p. *Math. Comput.*, 44:483–494, 1985. 46

[41] Ian F. Blake, Gadiel Seroussi, and Nigel Paul Smart. *Elliptic Curves in Cryptography*, volume 265 of *London Mathematical Society, Lecture Note Series*. Cambridge University Press, Cambridge, UK, 1999. 46

[42] A. J. Menezes. *Elliptic Curve Public Key Cryptosystems*. Kluwer Academic Publishers, Boston, MA, USA, 1993. 46, 47, 48

[43] ANSI. *Public Key Cryptography for the Financial Services Industry: Key Agreement and Key Transport Using Elliptic Curve Cryptography*. American National Standards Institute, National Bureau of Standards, ANSI X9.63, 2001. 47

[44] M. Abdalla, M. Bellare, and P. Rogaway. The oracle Diffie-Hellman assumptions and an analysis of DHIES. *Lecture Notes in Comput. Sci.*, 2020:143–158, 2001. 48